10.7.26

Monday, April 18, 2022 12:29 PM

```
#include <iostream>
#include <vector>
#include <cctype>
class King{
    public:
    int square;
    std::vector<char> legal_moves;
    bool color;
    bool hasmoved;
};
class Queen{
    public:
    int square;
    std::vector<char> legal_moves;
    bool color;
};
class Rook{
    public:
    int square;
    std::vector<char> legal_moves;
    bool color;
    bool hasmoved;
};
class Bishop{
    public:
    int square;
    std::vector<char> legal_moves;
    bool color;
};
class Knight{
    public:
    int square;
    std::vector<char> legal_moves;
    bool color;
};
class pawn{
    public:
    int square;
    std::vector<char> legal_moves;
    bool color;
    bool has_moved;
    bool en_passent_able;
};
void printboard(std::vector<char> board);
void FEN_to_position(std::vector<char> &board, std::string FEN);
std::string index_to_square(int index);
void piecelist(std::vector<char> &board, std::vector<pawn>
&whitePawns,std::vector<pawn> &blackPawns,std::vector<Bishop>
&whiteBishop, std::vector<Bishop> &blackBishop, std::vector<Knight>
&whiteKnight, std::vector<Knight> &blackKnight, std::vector<Rook>
&whiterook, std::vector<Rook> &blackrook, std::vector<Queen> &whiteQueen,
std::vector<Queen> &blackQueen, std::vector<King> &whiteKing,
std::vector<King> &blackKing);
void fillboard(std::vector<char> &board);
void Queenmovement(std::vector<char> &board, std::vector<Queen> &queens);
bool isblackpiece(std::vector<char> board, int square);
bool iswhitepiece(std::vector<char> board, int square);


int main(){
    std::vector<char> board; //rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR
w KQkq - 0 1
    std::vector<pawn> whitePawns;
    std::vector<pawn> blackPawns;
    std::vector<Bishop> whiteBishop;
    std::vector<Bishop> blackBishop;
    std::vector<Knight> whiteKnight;
    std::vector<Knight> blackKnight;
    std::vector<Rook> whiterook;
    std::vector<Rook> blackrook;
    std::vector<Queen> whiteQueen;
    std::vector<Queen> blackQueen;
    std::vector<King> whiteKing;
    std::vector<King> blackKing;
    fillboard(board);
    for(int i = 0; i < 64; i++){
        std::cout << i << " ";
        if(i % 8 == 7){
            std::cout << '\n';
        }
    }
    std::cout << index_to_square(17) << '\n';

    std::string FEN = "n1b3k1/4q1p1/3p4/p2Q4/Pp4Pp/7P/1PP5/R5K1 b - - 0
24";
    FEN_to_position(board, FEN);
    printboard(board);
    piecelist(board, whitePawns, blackPawns, whiteBishop, blackBishop,
whiteKnight, blackKnight, whiterook, blackrook, whiteQueen, blackQueen,
whiteKing, blackKing);
    Queenmovement(board, whiteQueen);
    printboard(whiteQueen[0].legal_moves);
    return 0;
}
void fillboard(std::vector<char> &board){
    for(int i = 0; i < 64; i++){
        board.push_back('0');
    }
}
void printboard(std::vector<char> board){
    for(int i = 0; i < 64; i++){
        std::cout << board[i];
        if(i % 8 == 7){
            std::cout << '\n';
        }
    }
    std::cout << '\n';
}
void FEN_to_position(std::vector<char> &board, std::string FEN){ //Bis
lang kann ich Stellungne transkribieren, jedoch beinhaltet der FEN-code
noch mehr Informationen, wie etwa wer am Zug ist, welcher Zug es ist, in
welche Richtungen man noch rochieren kann und wo en-passsant möglich ist.
Da diese Infos jedoch erst relevant werden sobald ich bei der
Zuggeneration bin, werde ich sie vorerst weg lassen
    int n = 0;
    for(int i = 0; i < FEN.length(); i++){
        if(FEN[i] == '/'){
            FEN.erase(FEN.begin() + i);
        }
    }
    for(int i = 0; i < FEN.length(); i++){
        if(std::isalpha(FEN[i]) && i+n < 64){ //std::isalpha() command von
ChatGPT
            board[i + n] = FEN[i];
        }
        if(std::isdigit(FEN[i])){
            n += FEN[i] - '0' - 1;
        }
        if(FEN[i] == ' '){
            return;
        }
    }
}
std::string index_to_square(int index){
    std::string square;
    std::string rank = std::to_string(-(index / 8) + 8); //Da wir bei FEN
und anderen nummereierungssysteme bei a8 beginnen zu zählen, stellt dies
eine Herausforderung dar für das Umrechnen einer Zahl zu einem Feld.
Deshalb diese komische Gleichung.
```

Ich hatte einen guten Start in mein Projekt denke ich. Jedoch wurde mit erst jetzt bewusst, wie gross das Projekt eigentlich sein wird. Es ist erst ein Tag vorbei und jetztschon habe ich 319 Zeilen code. Als erstes habe ich eine Funktion geschrieben, die mit FEN-codes zu einem Vektor umwandelt. Dies ist wichtig, da alle Stellungen als FEN-codes gespeichert sind. Diese Funktion ist jedoch noch nicht ganz komplett, da FEN-codes auch Informationen über Rochaderechte und en-passent enthalten. Da ich jedoch noch nicht so weit bin habe ich diese bislang weggelassen. Um die Stellung darzustellen habe ich mich für einen eindimensionalen Vektor entschieden. Ich habe einen super Artikel gefunden über das Darstellen von Stellungen
https://lichess.org/@/likeawizard/blog/review-of-different-board-representations-in-computer-chess/S9eQCAWa
In diesem Artikel stellte der Autor 3 Arten vor, um Stellungen darzustellen. Diese sind 2-dimensionale Arrays, eindimensionale Arrays und bitboards. Diese sind auch in dieser Reihenfolge die besten, laut dem Autor. Jedoch hat er auch erwähnt, dass jedes dieser Optionen einen Tausch zwischen Simplizität und Effizienz ist. Somit habe ich mich für eine Option entschieden, die nicht zu kompliziert ist aber auch nicht zu simple. Bitboards können nämlich eine ziemliche Hexerei sein. Dazu habe ich auch schon mit der Zuggeneration begonnen. Bislang habe ich zwar erst eine Richtung von Bewegung der weissen Dame fertig, jedoch habe ich damit den Blueprint für alle anderen Richtungen und muss nur noch die Variablen umstellen. Somit denke ich kann ich morgen die Bewegungen von Dame, Turm, Läufer und König fertig bekommen. Zumindest die simplen Bewegungen. Rochade werde ich noch machen müssen.

```
    int file = index % 8;
    std::cout << file << '\n';
    switch(file){
        case 0:
        square = "a" + rank;
        break;
        case 1:
        square = "b" + rank;
        break;
        case 2:
        square = "c" + rank;
        break;
        case 3:
        square = "d" + rank;
        break;
        case 4:
        square = "e" + rank;
        break;
        case 5:
        square = "f" + rank;
        break;
        case 6:
        square = "g" + rank;
        break;
        case 7:
        square = "h" + rank;
        break;
    }
    return square;
}
void piecelist(std::vector<char> &board, std::vector<pawn> &whitePawns,
std::vector<pawn> &blackPawns, std::vector<Bishop> &whiteBishop,
std::vector<Bishop> &blackBishop, std::vector<Knight> &whiteKnight,
std::vector<Knight> &blackKnight, std::vector<Rook> &whiterook,
std::vector<Rook> &blackrook, std::vector<Queen> &whiteQueen,
std::vector<Queen> &blackQueen, std::vector<King> &whiteKing,
std::vector<King> &blackKing){
    for(int i = 0; i < 64; i++){
        if(board[i] == 'p'){
            pawn pawn;
            pawn.color = 0;
            pawn.square = i;
            if(i / 8 == 1){
                pawn.has_moved = 0;
            }
            blackPawns.push_back(pawn);
        }
        if(board[i] == 'P'){
            pawn pawn;
            pawn.color = 1;
            pawn.square = i;
            if(i / 8 == 6){
                pawn.has_moved = 0;
            }
            whitePawns.push_back(pawn);
        }
        if(board[i] == 'n'){
            Knight knight;
            knight.color = 0;
            knight.square = i;
            blackKnight.push_back(knight);
        }
        if(board[i] == 'N'){
            Knight knight;
            knight.color = 1;
            knight.square = i;
            whiteKnight.push_back(knight);
        }
        if(board[i] == 'b'){
            Bishop bishop;
            bishop.color = 0;
            bishop.square = i;
            blackBishop.push_back(bishop);
        }
        if(board[i] == 'B'){
            Bishop bishop;
            bishop.color = 1;
            bishop.square = i;
            whiteBishop.push_back(bishop);
        }
        if(board[i] == 'r'){
            Rook rook;
            rook.color = 0;
            rook.square = i;
            blackrook.push_back(rook);
        }
        if(board[i] == 'R'){
            Rook rook;
            rook.color = 1;
            rook.square = i;
            whiterook.push_back(rook);
        }
        if(board[i] == 'q'){
            Queen queen;
            queen.color = 0;
            queen.square = i;
            blackQueen.push_back(queen);
        }
        if(board[i] == 'Q'){
            Queen queen;
            queen.color = 1;
            queen.square = i;
            whiteQueen.push_back(queen);
        }
        if(board[i] == 'k'){
            King king;
            king.color = 0;
            king.square = i;
            blackKing.push_back(king);
        }
        if(board[i] == 'K'){
            King king;
            king.color = 1;
            king.square = i;
            whiteKing.push_back(king);
        }
    }
}
void Queenmovement(std::vector<char> &board, std::vector<Queen> &queens){
    for(Queen &piece : queens){
        fillboard(piece.legal_moves);
        bool breaker = false;
        piece.legal_moves[piece.square] = 'Q';
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if(piece.square-(i*9) >= 0 && piece.square-((i-1)*9 % 8) !
= 0 && (board[piece.square - 9*i] == '0' || isblackpiece(board,
piece.square - 9*i))&& !iswhitepiece(board, piece.square - 9*i)){
                    piece.legal_moves[piece.square - i*9] = '1';
                    if(isblackpiece(board, piece.square - 9*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                    std::cout << "A" << '\n';
                }
                std::cout << "B" << '\n';
            }
        }
    }
}
bool isblackpiece(std::vector<char> board, int square){
    if(board[square] == 'p'){
        return true;
    }
```

```
    else if(board[square] == 'k'){
        return true;
    }
    else if(board[square] == 'n'){
        return true;
    }
    else if(board[square] == 'b'){
        return true;
    }
    else if(board[square] == 'r'){
        return true;
    }
    else if(board[square] == 'q'){
        return true;
    }
    else{
        return false;
    }
}
bool iswhitepiece(std::vector<char> board, int square){ //Hilfsfunktion
    if(board[square] == 'P'){
        return true;
    }
    else if(board[square] == 'K'){
        return true;
    }
    else if(board[square] == 'N'){
        return true;
    }
    else if(board[square] == 'B'){
        return true;
    }
    else if(board[square] == 'R'){
        return true;
    }
    else if(board[square] == 'Q'){
        return true;
    }
    else{
        return false;
    }
}
```

11.07.26

Samstag, 11. Juli 2026 21:48

```
#include <iostream>
#include <vector>
#include <cctype>
class King{
    public:
    int square;
    std::vector<char> legal_moves;
    bool color;
    bool hasmoved = true;
};
class Queen{
    public:
    int square;
    std::vector<char> legal_moves;
    bool color;
};
class Rook{
    public:
    int square;
    std::vector<char> legal_moves;
    bool color;
    bool hasmoved = true;
};
class Bishop{
    public:
    int square;
    std::vector<char> legal_moves;
    bool color;
};
class Knight{
    public:
    int square;
    std::vector<char> legal_moves;
    bool color;
};
class pawn{
    public:
    int square;
    std::vector<char> legal_moves;
    bool color;
    bool has_moved;
    bool en_passent_able = false;
};
void printboard(std::vector<char> board);
void FEN_to_position(std::vector<char> &board, std::string &FEN);
std::string index_to_square(int index);
void piecelist(std::vector<char> &board, std::vector<pawn>
&whitePawns,std::vector<pawn> &blackPawns,std::vector<Bishop>
&whiteBishop, std::vector<Bishop> &blackBishop, std::vector<Knight>
&whiteKnight, std::vector<Knight> &blackKnight, std::vector<Rook>
&whiterook, std::vector<Rook> &blackrook, std::vector<Queen> &whiteQueen,
std::vector<Queen> &blackQueen, std::vector<King> &whiteKing,
std::vector<King> &blackKing);
void fillboard(std::vector<char> &board);
void Queenmovement(std::vector<char> &board, std::vector<Queen> &queens,
std::vector<char> &legal_white_moves, std::vector<char>
&legal_black_moves);
bool isblackpiece(std::vector<char> board, int square);
bool iswhitepiece(std::vector<char> board, int square);
void Rookmovement(std::vector<char> &board, std::vector<Rook> &rooks,
std::vector<char> &legal_white_moves, std::vector<char>
&legal_black_moves);
void Bishopmovement(std::vector<char> &board, std::vector<Bishop>
&bishops, std::vector<char> &legal_white_moves, std::vector<char>
&legal_black_moves);
void Kingmovement(std::vector<char> &board, std::vector<King> &kings,
std::vector<char> &legal_white_moves, std::vector<char>
&legal_black_moves);
void legal_moves(std::vector<char> &board, std::vector<char>
&legal_white_moves, std::vector<char> &legal_black_moves,
std::vector<pawn> &whitePawns, std::vector<pawn> &blackPawns,
std::vector<Bishop> &whiteBishop, std::vector<Bishop> &blackBishop,
std::vector<Knight> &whiteKnight, std::vector<Knight> &blackKnight,
std::vector<Rook> &whiterook, std::vector<Rook> &blackrook,
std::vector<Queen> &whiteQueen, std::vector<Queen> &blackQueen,
std::vector<King> &whiteKing, std::vector<King> &blackKing);
void Knightmovement(std::vector<char> &board, std::vector<Knight>
&knights, std::vector<char> &legal_white_moves, std::vector<char>
&legal_black_moves);
void Whitepawnmovement(std::vector<char> &board, std::vector<pawn> &Pawn,
std::vector<char> &legal_white_moves);
void Blackpawnmovement(std::vector<char> &board, std::vector<pawn> &Pawn,
std::vector<char> &legal_black_moves);


int main(){
    std::vector<char> board; //rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR
w KQkq - 0 1
    std::vector<pawn> whitePawns;
    std::vector<pawn> blackPawns;
    std::vector<Bishop> whiteBishop;
    std::vector<Bishop> blackBishop;
    std::vector<Knight> whiteKnight;
    std::vector<Knight> blackKnight;
    std::vector<Rook> whiterook;
    std::vector<Rook> blackrook;
    std::vector<Queen> whiteQueen;
    std::vector<Queen> blackQueen;
    std::vector<King> whiteKing;
    std::vector<King> blackKing;
    std::vector<char> legal_black_moves;
    std::vector<char> legal_white_moves;
    bool turn; //true --> weiss am Zug, false --> schwarz am Zug
    fillboard(board);
    fillboard(legal_black_moves);
    fillboard(legal_white_moves);

    for(int i = 0; i < 64; i++){
        std::cout << i << " ";
        if(i % 8 == 7){
            std::cout << '\n';
        }
    }

    std::string FEN =
"r3k2r/1pq1bpp1/p2pbn1p/4p3/4P1P1/1PNQ3P/PBP2PB1/R3K2R w KQkq - 0 14";
    std::cout << FEN.length() << '\n';
    FEN_to_position(board, FEN);
    printboard(board);
    piecelist(board, whitePawns, blackPawns, whiteBishop, blackBishop,
whiteKnight, blackKnight, whiterook, blackrook, whiteQueen, blackQueen,
whiteKing, blackKing);
    legal_moves(board, legal_white_moves, legal_black_moves, whitePawns,
blackPawns, whiteBishop, blackBishop, whiteKnight, blackKnight, whiterook,
blackrook, whiteQueen, blackQueen, whiteKing, blackKing);
    printboard(legal_black_moves);
    return 0;
}
void fillboard(std::vector<char> &board){
    for(int i = 0; i < 64; i++){
        board.push_back('0');
    }
}
void printboard(std::vector<char> board){
    for(int i = 0; i < 64; i++){
        std::cout << board[i];
        if(i % 8 == 7){
            std::cout << '\n';
        }
    }
    std::cout << '\n';
}
void FEN_to_position(std::vector<char> &board, std::string &FEN){ //Bis
lang kann ich Stellungne transkribieren, jedoch beinhaltet der FEN-code
noch mehr Informationen, wie etwa wer am Zug ist, welcher Zug es ist, in
welche Richtungen man noch rochieren kann und wo en-passsant möglich ist.
Da diese Infos jedoch erst relevant werden sobald ich bei der
Zuggeneration bin, werde ich sie vorerst weg lassen
```

```
    int n = 0;
    int m = 0;
    for(int i = 0; i < FEN.length(); i++){
        if(FEN[i] == '/'){
            FEN.erase(FEN.begin() + i);
        }
    }
    for(int i = 0; i < FEN.length(); i++){
        if(std::isalpha(FEN[i]) && i+n < 64){ //std::isalpha() command von
ChatGPT
            board[i + n] = FEN[i];
        }
        if(std::isdigit(FEN[i])){
            n += FEN[i] - '0' - 1;
        }
        if(FEN[i] == ' '){
            break;
        }
    }
}
std::string index_to_square(int index){
    std::string square;
    std::string rank = std::to_string(-(index / 8) + 8); //Da wir bei FEN
und anderen nummereierungssysteme bei a8 beginnen zu zählen, stellt dies
eine Herausforderung dar für das Umrechnen einer Zahl zu einem Feld.
Deshalb diese komische Gleichung.
    int file = index % 8;
    std::cout << file << '\n';
    switch(file){
        case 0:
        square = "a" + rank;
        break;
        case 1:
        square = "b" + rank;
        break;
        case 2:
        square = "c" + rank;
        break;
        case 3:
        square = "d" + rank;
        break;
        case 4:
        square = "e" + rank;
        break;
        case 5:
        square = "f" + rank;
        break;
        case 6:
        square = "g" + rank;
        break;
        case 7:
        square = "h" + rank;
        break;
    }
    return square;
}
void piecelist(std::vector<char> &board, std::vector<pawn> &whitePawns,
std::vector<pawn> &blackPawns, std::vector<Bishop> &whiteBishop,
std::vector<Bishop> &blackBishop, std::vector<Knight> &whiteKnight,
std::vector<Knight> &blackKnight, std::vector<Rook> &whiterook,
std::vector<Rook> &blackrook, std::vector<Queen> &whiteQueen,
std::vector<Queen> &blackQueen, std::vector<King> &whiteKing,
std::vector<King> &blackKing){
    for(int i = 0; i < 64; i++){
        if(board[i] == 'p'){
            pawn pawn;
            pawn.color = 0;
            pawn.square = i;
            if(i / 8 == 1){
                pawn.has_moved = 0;
            }
            blackPawns.push_back(pawn);
        }
        if(board[i] == 'P'){
            pawn pawn;
            pawn.color = 1;
            pawn.square = i;
            if(i / 8 == 6){
                pawn.has_moved = 0;
            }
            whitePawns.push_back(pawn);
        }
        if(board[i] == 'n'){
            Knight knight;
            knight.color = 0;
            knight.square = i;
            blackKnight.push_back(knight);
        }
        if(board[i] == 'N'){
            Knight knight;
            knight.color = 1;
            knight.square = i;
            whiteKnight.push_back(knight);
        }
        if(board[i] == 'b'){
            Bishop bishop;
            bishop.color = 0;
            bishop.square = i;
            blackBishop.push_back(bishop);
        }
        if(board[i] == 'B'){
            Bishop bishop;
            bishop.color = 1;
            bishop.square = i;
            whiteBishop.push_back(bishop);
        }
        if(board[i] == 'r'){
            Rook rook;
            rook.color = 0;
            rook.square = i;
            blackrook.push_back(rook);
        }
        if(board[i] == 'R'){
            Rook rook;
            rook.color = 1;
            rook.square = i;
            whiterook.push_back(rook);
        }
        if(board[i] == 'q'){
            Queen queen;
            queen.color = 0;
            queen.square = i;
            blackQueen.push_back(queen);
        }
        if(board[i] == 'Q'){
            Queen queen;
            queen.color = 1;
            queen.square = i;
            whiteQueen.push_back(queen);
        }
        if(board[i] == 'k'){
            King king;
            king.color = 0;
            king.square = i;
            blackKing.push_back(king);
        }
        if(board[i] == 'K'){
            King king;
            king.color = 1;
            king.square = i;
            whiteKing.push_back(king);
        }
    }
}
void Queenmovement(std::vector<char> &board, std::vector<Queen> &queens,
std::vector<char> &legal_white_moves, std::vector<char>
&legal_black_moves){
    for(Queen &piece : queens){
        fillboard(piece.legal_moves);
        bool breaker = false;
        piece.legal_moves[piece.square] = 'Q';
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if(piece.square-(i*9) >= 0 && (piece.square-(i-1)*9) % 8 !
= 0 && (board[piece.square - 9*i] == '0' || isblackpiece(board,
```

Heute bin ich so weit mit dem generieren von legalen Zügen gekommen, dass ich nur noch die Rochade, Bauernumwandlungen sowie en-passent machen muss. Die Daten zu En-passent sowie zu der Rochade stehen im FEN-code, jedoch konnte habe ich noch keinen effizienten Weg gefunden, um sie zu extrahieren. Das Programm sieht zwar schon monströs gross aus mit seinen 1110 Zeilen, jedoch muss man berücksichtigen, dass das Programm nur so gross ist, da ich für jede Bewegungsrichtung für jede Figur eine eigene Bedingung sowie Konsequenz schreiben. Musste. Dies habe ich jedoch natürlich nur einmal gemacht und dann den Blueprint an die jeweiligen Figuren sowie Richtungen angepasst.

```
piece.square - 9*i))&& !iswhitepiece(board, piece.square - 9*i)){
                    piece.legal_moves[piece.square - i*9] = '1';
                    legal_white_moves[piece.square - i*9] = '1';
                    if(isblackpiece(board, piece.square - 9*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if(piece.square-(i*9) >= 0 && (piece.square-(i-1)*9 % 8) !
= 0 && (board[piece.square - 9*i] == '0' || iswhitepiece(board,
piece.square - 9*i))&& !isblackpiece(board, piece.square - 9*i)){
                    piece.legal_moves[piece.square - i*9] = '1';
                    legal_black_moves[piece.square - i*9] = '1';
                    if(iswhitepiece(board, piece.square - 9*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if(piece.square-(i*8) >= 0  && (board[piece.square - 8*i]
== '0' || isblackpiece(board, piece.square - 8*i))&& !iswhitepiece(board,
piece.square - 8*i)){
                    piece.legal_moves[piece.square - i*8] = '1';
                    legal_white_moves[piece.square - i*8] = '1';
                    if(isblackpiece(board, piece.square - 8*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if(piece.square-(i*8) >= 0  && (board[piece.square - 8*i]
== '0' || iswhitepiece(board, piece.square - 8*i))&& !isblackpiece(board,
piece.square - 8*i)){
                    piece.legal_moves[piece.square - i*8] = '1';
                    legal_black_moves[piece.square - i*8] = '1';
                    if(iswhitepiece(board, piece.square - 8*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if(piece.square-(i*7) >= 0 && (piece.square-(i-1)*7) % 8 !
= 7 && (board[piece.square - 7*i] == '0' || isblackpiece(board,
piece.square - 7*i))&& !iswhitepiece(board, piece.square - 7*i)){
                    piece.legal_moves[piece.square - i*7] = '1';
                    legal_white_moves[piece.square - i*7] = '1';
                    if(isblackpiece(board, piece.square - 7*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if(piece.square-(i*7) >= 0 && (piece.square-(i-1)*7) % 8 !
= 7 && (board[piece.square - 7*i] == '0' || iswhitepiece(board,
piece.square - 7*i))&& !isblackpiece(board, piece.square - 7*i)){
                    piece.legal_moves[piece.square - i*7] = '1';
                    legal_black_moves[piece.square - i*7] = '1';
                    if(iswhitepiece(board, piece.square - 7*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if((piece.square-(i-1)) % 8 != 0 && (board[piece.square -
i] == '0' || isblackpiece(board, piece.square - i))&& !iswhitepiece(board,
piece.square - i)){
                    piece.legal_moves[piece.square - i] = '1';
                    legal_white_moves[piece.square - i] = '1';
                    if(isblackpiece(board, piece.square - i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if((piece.square-(i-1)) % 8 != 0 && (board[piece.square -
i] == '0' || iswhitepiece(board, piece.square - i))&& !isblackpiece(board,
piece.square - i)){
                    piece.legal_moves[piece.square - i] = '1';
                    legal_black_moves[piece.square - i] = '1';
                    if(iswhitepiece(board, piece.square - i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if(piece.square+(i*7) <= 63 && (piece.square+(i-1)*7) %
8 != 0 && (board[piece.square + 7*i] == '0' || isblackpiece(board,
piece.square + 7*i))&& !iswhitepiece(board, piece.square + 7*i)){
                    piece.legal_moves[piece.square + i*7] = '1';
                    legal_white_moves[piece.square + i*7] = '1';
                    if(isblackpiece(board, piece.square + 7*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if(piece.square+(i*7) <= 63 && (piece.square+(i-1)*7) %
8 != 0 && (board[piece.square + 7*i] == '0' || iswhitepiece(board,
piece.square + 7*i))&& !isblackpiece(board, piece.square + 7*i)){
                    piece.legal_moves[piece.square + i*7] = '1';
                    legal_black_moves[piece.square + i*7] = '1';
                    if(iswhitepiece(board, piece.square + 7*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if(piece.square+(i*8) <= 63 && (board[piece.square + 8*i]
== '0' || isblackpiece(board, piece.square + 8*i))&& !iswhitepiece(board,
```

```
piece.square + 8*i)){
                    piece.legal_moves[piece.square + i*8] = '1';
                    legal_white_moves[piece.square + i*8] = '1';
                    if(isblackpiece(board, piece.square + 8*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if(piece.square+(i*8) <= 63 && (board[piece.square + 8*i]
== '0' || iswhitepiece(board, piece.square + 8*i))&& !isblackpiece(board,
piece.square + 8*i)){
                    piece.legal_moves[piece.square + i*8] = '1';
                    legal_black_moves[piece.square + i*8] = '1';
                    if(iswhitepiece(board, piece.square + 8*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if(piece.square+(i*9) <= 63 && (piece.square+(i-1)*9) %
8 != 7 && (board[piece.square + 9*i] == '0' || isblackpiece(board,
piece.square + 9*i))&& !iswhitepiece(board, piece.square + 9*i)){
                    piece.legal_moves[piece.square + i*9] = '1';
                    legal_white_moves[piece.square + i*9] = '1';
                    if(isblackpiece(board, piece.square + 9*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if(piece.square+(i*9) <= 63 && (piece.square+(i-1)*9) %
8 != 7 && (board[piece.square + 9*i] == '0' || iswhitepiece(board,
piece.square + 9*i))&& !isblackpiece(board, piece.square + 9*i)){
                    piece.legal_moves[piece.square + i*9] = '1';
                    legal_black_moves[piece.square + i*9] = '1';
                    if(iswhitepiece(board, piece.square + 9*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if((piece.square+(i-1)) % 8 != 7 && (board[piece.square +
i] == '0' || isblackpiece(board, piece.square + i))&& !iswhitepiece(board,
piece.square + i)){
                    piece.legal_moves[piece.square + i] = '1';
                    legal_white_moves[piece.square + i] = '1';
                    if(isblackpiece(board, piece.square + i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if((piece.square+(i-1)) % 8 != 7 && (board[piece.square +
i] == '0' || iswhitepiece(board, piece.square + i))&& !isblackpiece(board,
piece.square + i)){
                    piece.legal_moves[piece.square + i] = '1';
                    legal_black_moves[piece.square + i] = '1';
                    if(iswhitepiece(board, piece.square + i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
    }
}
bool isblackpiece(std::vector<char> board, int square){
    if(board[square] == 'p'){
        return true;
    }
    else if(board[square] == 'k'){
        return true;
    }
    else if(board[square] == 'n'){
        return true;
    }
    else if(board[square] == 'b'){
        return true;
    }
    else if(board[square] == 'r'){
        return true;
    }
    else if(board[square] == 'q'){
        return true;
    }
    else{
        return false;
    }
}
bool iswhitepiece(std::vector<char> board, int square){ //Hilfsfunktion
    if(board[square] == 'P'){
        return true;
    }
    else if(board[square] == 'K'){
        return true;
    }
    else if(board[square] == 'N'){
        return true;
    }
    else if(board[square] == 'B'){
        return true;
    }
    else if(board[square] == 'R'){
        return true;
    }
    else if(board[square] == 'Q'){
        return true;
    }
    else{
        return false;
    }
}
void Rookmovement(std::vector<char> &board, std::vector<Rook> &rooks,
std::vector<char> &legal_white_moves, std::vector<char>
&legal_black_moves){
    for(Rook &piece : rooks){
        fillboard(piece.legal_moves);
        bool breaker = false;
        piece.legal_moves[piece.square] = 'R';
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if(piece.square-(i*8) >= 0  && (board[piece.square - 8*i]
== '0' || isblackpiece(board, piece.square - 8*i))&& !iswhitepiece(board,
piece.square - 8*i)){
                    piece.legal_moves[piece.square - i*8] = '1';
                    legal_white_moves[piece.square - i*8] = '1';
                    if(isblackpiece(board, piece.square - 8*i)){
                        breaker = true;
                    }
                }
```

```
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if(piece.square-(i*8) >= 0  && (board[piece.square - 8*i]
== '0' || iswhitepiece(board, piece.square - 8*i))&& !isblackpiece(board,
piece.square - 8*i)){
                    piece.legal_moves[piece.square - i*8] = '1';
                    legal_black_moves[piece.square - i*8] = '1';
                    if(iswhitepiece(board, piece.square - 8*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if((piece.square-(i-1)) % 8 != 0 && (board[piece.square -
i] == '0' || isblackpiece(board, piece.square - i))&& !iswhitepiece(board,
piece.square - i)){
                    piece.legal_moves[piece.square - i] = '1';
                    legal_white_moves[piece.square - i] = '1';
                    if(isblackpiece(board, piece.square - i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if((piece.square-(i-1)) % 8 != 0 && (board[piece.square -
i] == '0' || iswhitepiece(board, piece.square - i))&& !isblackpiece(board,
piece.square - i)){
                    piece.legal_moves[piece.square - i] = '1';
                    legal_black_moves[piece.square - i] = '1';
                    if(iswhitepiece(board, piece.square - i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if(piece.square+(i*8) <= 63 && (board[piece.square + 8*i]
== '0' || isblackpiece(board, piece.square + 8*i))&& !iswhitepiece(board,
piece.square + 8*i)){
                    piece.legal_moves[piece.square + i*8] = '1';
                    legal_white_moves[piece.square + i*8] = '1';
                    if(isblackpiece(board, piece.square + 8*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if(piece.square+(i*8) <= 63 && (board[piece.square + 8*i]
== '0' || iswhitepiece(board, piece.square + 8*i))&& !isblackpiece(board,
piece.square + 8*i)){
                    piece.legal_moves[piece.square + i*8] = '1';
                    legal_black_moves[piece.square + i*8] = '1';
                    if(iswhitepiece(board, piece.square + 8*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if((piece.square+(i-1)) % 8 != 7 && (board[piece.square +
i] == '0' || isblackpiece(board, piece.square + i))&& !iswhitepiece(board,
piece.square + i)){
                    piece.legal_moves[piece.square + i] = '1';
                    legal_white_moves[piece.square + i] = '1';
                    if(isblackpiece(board, piece.square + i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if((piece.square+(i-1)) % 8 != 7 && (board[piece.square +
i] == '0' || iswhitepiece(board, piece.square + i))&& !isblackpiece(board,
piece.square + i)){
                    piece.legal_moves[piece.square + i] = '1';
                    legal_black_moves[piece.square + i] = '1';
                    if(iswhitepiece(board, piece.square + i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
    }
}
void Bishopmovement(std::vector<char> &board, std::vector<Bishop>
&bishops, std::vector<char> & legal_white_moves, std::vector<char>
&legal_black_moves){
    for(Bishop &piece : bishops){
        fillboard(piece.legal_moves);
        bool breaker = false;
        piece.legal_moves[piece.square] = 'B';
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if(piece.square-(i*9) >= 0 && (piece.square-(i-1)*9) % 8 !
= 0 && (board[piece.square - 9*i] == '0' || isblackpiece(board,
piece.square - 9*i))&& !iswhitepiece(board, piece.square - 9*i)){
                    piece.legal_moves[piece.square - i*9] = '1';
                    legal_white_moves[piece.square - i*9] = '1';
                    if(isblackpiece(board, piece.square - 9*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if(piece.square-(i*9) >= 0 && (piece.square-(i-1)*9 % 8) !
= 0 && (board[piece.square - 9*i] == '0' || iswhitepiece(board,
piece.square - 9*i))&& !isblackpiece(board, piece.square - 9*i)){
                    piece.legal_moves[piece.square - i*9] = '1';
                    legal_black_moves[piece.square - i*9] = '1';
                    if(iswhitepiece(board, piece.square - 9*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
```

```
                if(piece.square-(i*7) >= 0 && (piece.square-(i-1)*7) % 8 !
= 7 && (board[piece.square - 7*i] == '0' || isblackpiece(board,
piece.square - 7*i))&& !iswhitepiece(board, piece.square - 7*i)){
                    piece.legal_moves[piece.square - i*7] = '1';
                    legal_white_moves[piece.square - i*7] = '1';
                    if(isblackpiece(board, piece.square - 7*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if(piece.square-(i*7) >= 0 && (piece.square-(i-1)*7) % 8 !
= 7 && (board[piece.square - 7*i] == '0' || iswhitepiece(board,
piece.square - 7*i))&& !isblackpiece(board, piece.square - 7*i)){
                    piece.legal_moves[piece.square - i*7] = '1';
                    legal_black_moves[piece.square - i*7] = '1';
                    if(iswhitepiece(board, piece.square - 7*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if(piece.square+(i*7) <= 63 && (piece.square+(i-1)*7) %
8 != 0 && (board[piece.square + 7*i] == '0' || isblackpiece(board,
piece.square + 7*i))&& !iswhitepiece(board, piece.square + 7*i)){
                    piece.legal_moves[piece.square + i*7] = '1';
                    legal_white_moves[piece.square + i*7] = '1';
                    if(isblackpiece(board, piece.square + 7*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if(piece.square+(i*7) <= 63 && (piece.square+(i-1)*7) %
8 != 0 && (board[piece.square + 7*i] == '0' || iswhitepiece(board,
piece.square + 7*i))&& !isblackpiece(board, piece.square + 7*i)){
                    piece.legal_moves[piece.square + i*7] = '1';
                    legal_black_moves[piece.square + i*7] = '1';
                    if(iswhitepiece(board, piece.square + 7*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
        for(int i = 1; breaker == false; i++){
            if(piece.color == 1){
                if(piece.square+(i*9) <= 63 && (piece.square+(i-1)*9) %
8 != 7 && (board[piece.square + 9*i] == '0' || isblackpiece(board,
piece.square + 9*i))&& !iswhitepiece(board, piece.square + 9*i)){
                    piece.legal_moves[piece.square + i*9] = '1';
                    legal_white_moves[piece.square + i*9] = '1';
                    if(isblackpiece(board, piece.square + 9*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
            if(piece.color == 0){
                if(piece.square+(i*9) <= 63 && (piece.square+(i-1)*9) %
8 != 7 && (board[piece.square + 9*i] == '0' || iswhitepiece(board,
piece.square + 9*i))&& !isblackpiece(board, piece.square + 9*i)){
                    piece.legal_moves[piece.square + i*9] = '1';
                    legal_black_moves[piece.square + i*9] = '1';
                    if(iswhitepiece(board, piece.square + 9*i)){
                        breaker = true;
                    }
                }
                else{
                    breaker = true;
                }
            }
        }
        breaker = false;
    }
}
void Kingmovement(std::vector<char> &board, std::vector<King> &kings,
std::vector<char> &legal_white_moves, std::vector<char>
&legal_black_moves){
    for(King &piece : kings){ //Rochade muss ich noch machen
        fillboard(piece.legal_moves);
        piece.legal_moves[piece.square] = 'K';
            if(piece.color == 1){
                if(piece.square-9 >= 0 && piece.square % 8 != 0 &&
(board[piece.square - 9] == '0' || isblackpiece(board, piece.square -
9))&& !iswhitepiece(board, piece.square - 9) &&
legal_black_moves[piece.square - 9] != '1'){
                    piece.legal_moves[piece.square - 9] = '1';
                    legal_white_moves[piece.square - 9] = '1';
                    if(isblackpiece(board, piece.square - 9)){
                    }
                }
            }
            if(piece.color == 0){
                if(piece.square-9 >= 0 && piece.square % 8 != 0 &&
(board[piece.square - 9] == '0' || iswhitepiece(board, piece.square -
9))&& !isblackpiece(board, piece.square - 9) &&
legal_white_moves[piece.square - 9] != '1'){
                    piece.legal_moves[piece.square - 9] = '1';
                    legal_black_moves[piece.square - 9] = '1';
                    if(iswhitepiece(board, piece.square - 9)){
                    }
                }
            }
            if(piece.color == 1){
                if(piece.square-8 >= 0  && (board[piece.square - 8] == '0'
|| isblackpiece(board, piece.square - 8))&& !iswhitepiece(board,
piece.square - 8) && legal_black_moves[piece.square - 8] != '1'){
                    piece.legal_moves[piece.square - 8] = '1';
                    legal_white_moves[piece.square - 8] = '1';
                    if(isblackpiece(board, piece.square - 8)){
                    }
                }
            }
            if(piece.color == 0){
                if(piece.square-8 >= 0  && (board[piece.square - 8] == '0'
|| iswhitepiece(board, piece.square - 8))&& !isblackpiece(board,
piece.square - 8) && legal_white_moves[piece.square - 8] != '1'){
                    piece.legal_moves[piece.square - 8] = '1';
                    legal_black_moves[piece.square - 8] = '1';
                    if(iswhitepiece(board, piece.square - 8)){
                    }
                }
            }
            if(piece.color == 1){
                if(piece.square-7 >= 0 && piece.square % 8 != 7 &&
(board[piece.square - 7] == '0' || isblackpiece(board, piece.square -
7))&& !iswhitepiece(board, piece.square - 7) &&
legal_black_moves[piece.square - 7] != '1'){
                    piece.legal_moves[piece.square - 7] = '1';
                    legal_white_moves[piece.square - 7] = '1';
                    if(isblackpiece(board, piece.square - 7)){
                    }
                }
            }
            if(piece.color == 0){
                if(piece.square-7 >= 0 && piece.square % 8 != 7 &&
(board[piece.square - 7] == '0' || iswhitepiece(board, piece.square -
```

```
7))&& !isblackpiece(board, piece.square - 7) &&
legal_white_moves[piece.square - 7] != '1'){
                    piece.legal_moves[piece.square - 7] = '1';
                    legal_black_moves[piece.square - 7] = '1';
                    if(iswhitepiece(board, piece.square - 7)){
                    }
                }
            }
            if(piece.color == 1){
                if(piece.square % 8 != 0 && (board[piece.square - 1] ==
'0' || isblackpiece(board, piece.square - 1))&& !iswhitepiece(board,
piece.square - 1) && legal_black_moves[piece.square - 1] != '1'){
                    piece.legal_moves[piece.square - 1] = '1';
                    legal_white_moves[piece.square - 1] = '1';
                    if(isblackpiece(board, piece.square - 1)){
                    }
                }
            }
            if(piece.color == 0){
                if(piece.square % 8 != 0 && (board[piece.square - 1] ==
'0' || iswhitepiece(board, piece.square - 1))&& !isblackpiece(board,
piece.square - 1) && legal_white_moves[piece.square - 1] != '1'){
                    piece.legal_moves[piece.square - 1] = '1';
                    legal_black_moves[piece.square - 1] = '1';
                    if(iswhitepiece(board, piece.square - 1)){
                    }
                }
            }

            if(piece.color == 1){
                if(piece.square+7 <= 63 && piece.square % 8 != 0 &&
(board[piece.square + 7] == '0' || isblackpiece(board, piece.square +
7))&& !iswhitepiece(board, piece.square + 7) &&
legal_black_moves[piece.square + 7] != '1'){
                    piece.legal_moves[piece.square + 7] = '1';
                    legal_white_moves[piece.square + 7] = '1';
                    if(isblackpiece(board, piece.square + 7)){
                    }
                }
            }
            if(piece.color == 0){
                if(piece.square+7 <= 63 && piece.square % 8 != 0 &&
(board[piece.square + 7] == '0' || iswhitepiece(board, piece.square +
7))&& !isblackpiece(board, piece.square + 7) &&
legal_white_moves[piece.square + 7] != '1'){
                    piece.legal_moves[piece.square + 7] = '1';
                    legal_black_moves[piece.square + 7] = '1';
                    if(iswhitepiece(board, piece.square + 7)){
                    }
                }
            }

            if(piece.color == 1){
                if(piece.square+8 <= 63 && (board[piece.square + 8] == '0'
|| isblackpiece(board, piece.square + 8))&& !iswhitepiece(board,
piece.square + 8) && legal_black_moves[piece.square + 8] != '1'){
                    piece.legal_moves[piece.square + 8] = '1';
                    legal_white_moves[piece.square + 8] = '1';
                    if(isblackpiece(board, piece.square + 8)){
                    }
                }
            }
            if(piece.color == 0){
                if(piece.square+8 <= 63 && (board[piece.square + 8] == '0'
|| iswhitepiece(board, piece.square + 8))&& !isblackpiece(board,
piece.square + 8) && legal_white_moves[piece.square + 8] != '1'){
                    piece.legal_moves[piece.square + 8] = '1';
                    legal_black_moves[piece.square + 8] = '1';
                    if(iswhitepiece(board, piece.square + 8)){
                    }
                }
            }

            if(piece.color == 1){
                if(piece.square+9 <= 63 && piece.square % 8 != 7 &&
(board[piece.square + 9] == '0' || isblackpiece(board, piece.square +
9))&& !iswhitepiece(board, piece.square + 9) &&
legal_black_moves[piece.square + 9] != '1'){
                    piece.legal_moves[piece.square + 9] = '1';
                    legal_white_moves[piece.square + 9] = '1';
                    if(isblackpiece(board, piece.square + 9)){
                    }
                }
            }
            if(piece.color == 0){
                if(piece.square+9 <= 63 && piece.square % 8 != 7 &&
(board[piece.square + 9] == '0' || iswhitepiece(board, piece.square +
9))&& !isblackpiece(board, piece.square + 9) &&
legal_white_moves[piece.square + 9] != '1'){
                    piece.legal_moves[piece.square + 9] = '1';
                    legal_black_moves[piece.square + 9] = '1';
                    if(iswhitepiece(board, piece.square + 9)){
                    }
                }
            }

            if(piece.color == 1){
                if(piece.square % 8 != 7 && (board[piece.square + 1] ==
'0' || isblackpiece(board, piece.square + 1))&& !iswhitepiece(board,
piece.square + 1) && legal_black_moves[piece.square + 1] != '1'){
                    piece.legal_moves[piece.square + 1] = '1';
                    legal_white_moves[piece.square + 1] = '1';
                    if(isblackpiece(board, piece.square + 1)){
                    }
                }
            }
            if(piece.color == 0){
                if(piece.square % 8 != 7 && (board[piece.square + 1] ==
'0' || iswhitepiece(board, piece.square + 1))&& !isblackpiece(board,
piece.square + 1) && legal_white_moves[piece.square + 1] != '1'){
                    piece.legal_moves[piece.square + 1] = '1';
                    legal_black_moves[piece.square + 1] = '1';
                    if(iswhitepiece(board, piece.square + 1)){
                    }
                }
            }
    }
}
void Knightmovement(std::vector<char> &board, std::vector<Knight>
&knights, std::vector<char> &legal_white_moves, std::vector<char>
&legal_black_moves){
    for(Knight &piece : knights){
        fillboard(piece.legal_moves);
        if(piece.color = 1){
            if(piece.square - 10 >= 0 && piece.square % 8 > 1
&&(board[piece.square - 10] == '0' || isblackpiece(board, piece.square -
10)) && !iswhitepiece(board, piece.square - 10)){
                piece.legal_moves[piece.square - 10] = '1';
                legal_white_moves[piece.square - 10] = '1';
            }
        }
        if(piece.color = 0){
            if(piece.square - 10 >= 0 && piece.square % 8 > 1
&&(board[piece.square - 10] == '0' || iswhitepiece(board, piece.square -
10)) && !isblackpiece(board, piece.square - 10)){
                piece.legal_moves[piece.square - 10] = '1';
                legal_black_moves[piece.square - 10] = '1';
            }
        }
        if(piece.color = 1){
            if(piece.square - 17 >= 0 && piece.square % 8 > 0
&&(board[piece.square - 17] == '0' || isblackpiece(board, piece.square -
17)) && !iswhitepiece(board, piece.square - 17)){
                piece.legal_moves[piece.square - 17] = '1';
                legal_white_moves[piece.square - 17] = '1';
            }
        }
        if(piece.color = 0){
            if(piece.square - 17 >= 0 && piece.square % 8 > 0
&&(board[piece.square - 17] == '0' || iswhitepiece(board, piece.square -
17)) && !isblackpiece(board, piece.square - 17)){
                piece.legal_moves[piece.square - 17] = '1';
                legal_black_moves[piece.square - 17] = '1';
            }
        }
```

```
        if(piece.color = 1){
            if(piece.square - 15 >= 0 && piece.square % 8 < 7
&&(board[piece.square - 15] == '0' || isblackpiece(board, piece.square -
15)) && !iswhitepiece(board, piece.square - 15)){
                piece.legal_moves[piece.square - 15] = '1';
                legal_white_moves[piece.square - 15] = '1';
            }
        }
        if(piece.color = 0){
            if(piece.square - 15 >= 0 && piece.square % 8 < 7
&&(board[piece.square - 15] == '0' || iswhitepiece(board, piece.square -
15)) && !isblackpiece(board, piece.square - 15)){
                piece.legal_moves[piece.square - 15] = '1';
                legal_black_moves[piece.square - 15] = '1';
            }
        }
        if(piece.color = 1){
            if(piece.square - 6 >= 0 && piece.square % 8 < 6
&&(board[piece.square - 6] == '0' || isblackpiece(board, piece.square -
6)) && !iswhitepiece(board, piece.square - 6)){
                piece.legal_moves[piece.square - 6] = '1';
                legal_white_moves[piece.square - 6] = '1';
            }
        }
        if(piece.color = 0){
            if(piece.square - 6 >= 0 && piece.square % 8 < 6
&&(board[piece.square - 6] == '0' || iswhitepiece(board, piece.square -
6)) && !isblackpiece(board, piece.square - 6)){
                piece.legal_moves[piece.square - 6] = '1';
                legal_black_moves[piece.square - 6] = '1';
            }
        }
        if(piece.color = 1){
            if(piece.square + 10 <= 63 && piece.square % 8 < 6
&&(board[piece.square + 10] == '0' || isblackpiece(board, piece.square +
10)) && !iswhitepiece(board, piece.square + 10)){
                piece.legal_moves[piece.square + 10] = '1';
                legal_white_moves[piece.square + 10] = '1';
            }
        }
        if(piece.color = 0){
            if(piece.square + 10 <= 63 && piece.square % 8 < 6
&&(board[piece.square + 10] == '0' || iswhitepiece(board, piece.square +
10)) && !isblackpiece(board, piece.square + 10)){
                piece.legal_moves[piece.square + 10] = '1';
                legal_black_moves[piece.square + 10] = '1';
            }
        }
        if(piece.color = 1){
            if(piece.square + 17 <= 63 && piece.square % 8 < 7
&&(board[piece.square + 17] == '0' || isblackpiece(board, piece.square +
17)) && !iswhitepiece(board, piece.square + 17)){
                piece.legal_moves[piece.square + 17] = '1';
                legal_white_moves[piece.square + 17] = '1';
            }
        }
        if(piece.color = 0){
            if(piece.square + 17 <= 63 && piece.square % 8 < 7
&&(board[piece.square + 17] == '0' || iswhitepiece(board, piece.square +
17)) && !isblackpiece(board, piece.square + 17)){
                piece.legal_moves[piece.square + 17] = '1';
                legal_black_moves[piece.square + 17] = '1';
            }
        }
        if(piece.color = 1){
            if(piece.square + 15 <= 63 && piece.square % 8 > 0
&&(board[piece.square + 15] == '0' || isblackpiece(board, piece.square +
15)) && !iswhitepiece(board, piece.square + 15)){
                piece.legal_moves[piece.square + 15] = '1';
                legal_white_moves[piece.square + 15] = '1';
            }
        }
        if(piece.color = 0){
            if(piece.square + 15 <= 63 && piece.square % 8 > 0
&&(board[piece.square + 15] == '0' || iswhitepiece(board, piece.square +
15)) && !isblackpiece(board, piece.square + 15)){
                piece.legal_moves[piece.square + 15] = '1';
                legal_black_moves[piece.square + 15] = '1';
            }
        }
        if(piece.color = 1){
            if(piece.square + 6 <= 63 && piece.square % 8 > 1
&&(board[piece.square + 6] == '0' || isblackpiece(board, piece.square +
6)) && !iswhitepiece(board, piece.square + 6)){
                piece.legal_moves[piece.square + 6] = '1';
                legal_white_moves[piece.square + 6] = '1';
            }
        }
        if(piece.color = 0){
            if(piece.square + 6 <= 63 && piece.square % 8 > 1
&&(board[piece.square + 6] == '0' || iswhitepiece(board, piece.square +
6)) && !isblackpiece(board, piece.square + 6)){
                piece.legal_moves[piece.square + 6] = '1';
                legal_black_moves[piece.square + 6] = '1';
            }
        }
    }
}
void Whitepawnmovement(std::vector<char> &board, std::vector<pawn> &Pawn,
std::vector<char> &legal_white_moves){
    for(pawn &pawn : Pawn){
        fillboard(pawn.legal_moves);
        pawn.legal_moves[pawn.square] = 'P';
        if(board[pawn.square - 8] == '0'){
            pawn.legal_moves[pawn.square - 8] = '1';
            legal_white_moves[pawn.square - 8] = '1';
        }
        if(board[pawn.square - 8] == '0' && board[pawn.square - 16] == '0'
&& (pawn.square / 8) == 6){
            pawn.legal_moves[pawn.square - 16] = '1';
            legal_white_moves[pawn.square - 16] = '1';
        }
        if(pawn.square % 8 != 0 && isblackpiece(board, pawn.square - 9)){
            pawn.legal_moves[pawn.square - 9] = '1';
            legal_white_moves[pawn.square - 9] = '1';
        }
        if(pawn.square % 8 != 7 && isblackpiece(board, pawn.square - 7)){
            pawn.legal_moves[pawn.square - 7] = '1';
            legal_white_moves[pawn.square - 7] = '1';
        }
    }
}
void Blackpawnmovement(std::vector<char> &board, std::vector<pawn> &Pawn,
std::vector<char> &legal_black_moves){
    for(pawn &pawn : Pawn){
        fillboard(pawn.legal_moves);
        pawn.legal_moves[pawn.square] = 'P';
        if(board[pawn.square + 8] == '0'){
            pawn.legal_moves[pawn.square + 8] = '1';
            legal_black_moves[pawn.square + 8] = '1';
        }
        if(board[pawn.square + 8] == '0' && board[pawn.square + 16] == '0'
&& (pawn.square / 8) == 1){
            pawn.legal_moves[pawn.square + 16] = '1';
            legal_black_moves[pawn.square + 16] = '1';
        }
        if(pawn.square % 8 != 0 && iswhitepiece(board, pawn.square + 7)){
            pawn.legal_moves[pawn.square + 7] = '1';
            legal_black_moves[pawn.square + 7] = '1';
        }
        if(pawn.square % 8 != 7 && iswhitepiece(board, pawn.square + 9)){
            pawn.legal_moves[pawn.square + 9] = '1';
            legal_black_moves[pawn.square + 9] = '1';
        }
    }
}
void legal_moves(std::vector<char> &board, std::vector<char>
&legal_white_moves, std::vector<char> &legal_black_moves,
std::vector<pawn> &whitePawns, std::vector<pawn> &blackPawns,
std::vector<Bishop> &whiteBishop, std::vector<Bishop> &blackBishop,
std::vector<Knight> &whiteKnight, std::vector<Knight> &blackKnight,
std::vector<Rook> &whiterook, std::vector<Rook> &blackrook,
std::vector<Queen> &whiteQueen, std::vector<Queen> &blackQueen,
std::vector<King> &whiteKing, std::vector<King> &blackKing){
    Queenmovement(board, whiteQueen, legal_white_moves,
```

```
legal_black_moves);
    Queenmovement(board, blackQueen, legal_white_moves,
legal_black_moves);
    Rookmovement(board, whiterook, legal_white_moves, legal_black_moves);
    Rookmovement(board, blackrook, legal_white_moves, legal_black_moves);
    Bishopmovement(board, whiteBishop, legal_white_moves,
legal_black_moves);
    Bishopmovement(board, blackBishop, legal_white_moves,
legal_black_moves);
    Kingmovement(board, whiteKing, legal_white_moves, legal_black_moves);
    Kingmovement(board, blackKing, legal_white_moves, legal_black_moves);
    Knightmovement(board, whiteKnight, legal_white_moves,
legal_black_moves);
    Knightmovement(board, blackKnight, legal_white_moves,
legal_black_moves);
    Whitepawnmovement(board, whitePawns, legal_white_moves);
    Blackpawnmovement(board, blackPawns, legal_black_moves);
}
void FEN_extra(std::vector<char> &board, std::string FEN, bool &turn,
std::vector<char> &legal_white_moves, std::vector<char>
&legal_black_moves, std::vector<King> whiteking, std::vector<King>
blackking, std::vector<Rook> whiterook, std::vector<Rook> blackrook,
std::vector<pawn> whitepawn, std::vector<pawn> blackpawn){
    int n = 0; //Funktion ist tendenzeill unnütig. Idee für nächstes
Bearbeiten: den String bei " " auseinanderschneiden, um die Abschnitte
separat bearbeiten zu können.
    for(int i = 0; i < FEN.length(); i++){
        if (FEN[i] == ' ');
        n = i;
    }
    for(int i = n; i < FEN.length(); i++){
        if (FEN[i] == 'w'){
            turn = true;
        }
        if(FEN[i] == 'b'){
            turn = false;
        }
        if(FEN[i] == 'K'){
            whiteking[0].hasmoved = false;
            whiterook[rookfinder(board, whiterook, 63)].hasmoved = false;
//vielleicht irgendwann mal überarbeiten. Out-of-bounds Gefahr
        }
        if(FEN[i] == 'Q'){
            whiteking[0].hasmoved = false;
            whiterook[rookfinder(board, whiterook, 56)].hasmoved = false;
        }
        if(FEN[i] == 'k'){
            blackking[0].hasmoved = false;
            blackrook[rookfinder(board, whiterook, 63)].hasmoved = false;
        }
        if(FEN[i] == 'q'){
            blackking[0].hasmoved = false;
            blackrook[rookfinder(board, whiterook, 56)].hasmoved = false;
        }
    }
}
int rookfinder(std::vector<char> &board, std::vector<Rook> rooks, int
square){
    for(int i = 0; i < rooks.size(); i++){
        if(rooks[i].square == square){
            return i;
        }
        else{
            return -1;
        }
    }
}
```

19.07.26

Montag, 20. Juli 2026 10:19

Ich habe gestern eine Funktion gebaut, die angibt, wie viele legale Züge in einer gegebenen Stellung ist. Dies funktioniert gut, bis auf Szenarien, wo ein König im Schach steht oder eine Figur gefesselt ist. Somit ist mein nächster Schritt, jene Züge zu verbieten, wonach der eigene König im Schach stehen würde.

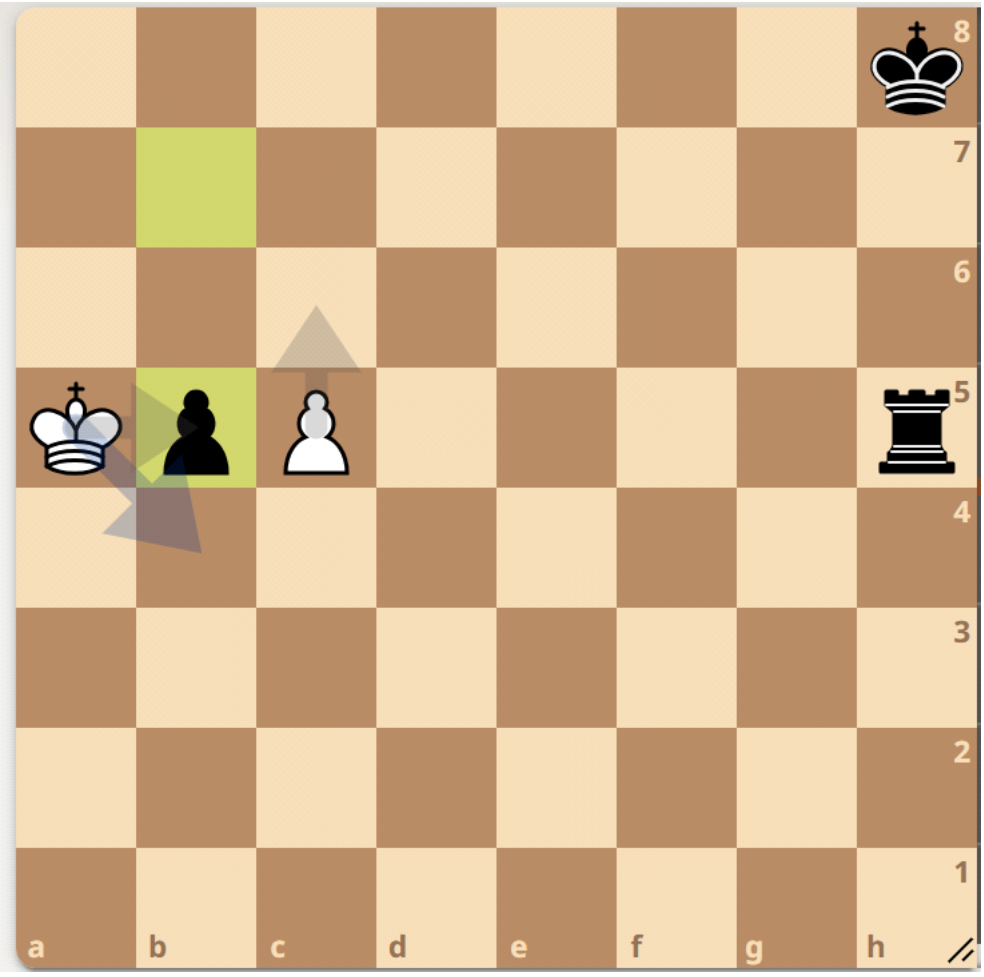
24.07.26

Freitag, 24. Juli 2026 22:35

Um die Zuggeneration abzuschliessen muss ich noch jene Züge herausfiltern, welche aufgrund von Schachs illegal sind. Dafür habe ich eine Funktion gebaut:

```
bool check_check_white(std::vector<char> board, int startingsquare, int
targetsquare, std::vector<char> legal_white_moves, std::vector<char>
legal_black_moves, std::vector<King> whiteKing, std::vector<pawn> blackPawns,
std::vector<Bishop> blackBishop, std::vector<Knight> blackKnight,
std::vector<Rook> blackrook, std::vector<Queen> blackQueen, std::vector<King>
blackKing, std::vector<pawn> whitePawns, std::vector<Bishop> whiteBishop,
std::vector<Knight> whiteKnight, std::vector<Rook> whiterook,
std::vector<Queen> whiteQueen){
    char temp = board[startingsquare];
    board[targetsquare] = temp;
    board[startingsquare] = '0';
    piecelist(board, whitePawns, blackPawns, whiteBishop, blackBishop,
whiteKnight, blackKnight, whiterook, blackrook, whiteQueen, blackQueen,
whiteKing, blackKing);
    legal_moves_black(board, legal_white_moves, legal_black_moves, blackPawns,
blackBishop, blackKnight, blackrook, blackQueen, blackKing);
    printboard(board);
    printboard(blackBishop[0].legal_moves);
    if(legal_black_moves[whiteKing[0].square] == '1'){
        return true;
    }
    return false;
}
```

Jedoch hat sie das Problem, dass es die legalen Züge aus irgendeine Grund nicht bearbeitet. Ich hatte lange Zeit Probleme dabei, eine gute Idee zu finden für genau das. Ich wollte nämlich die Option, wobei ich den Zug durchführe und dann nach Schach prüfe vermeiden, da er ziemlich Rechenintensiv werden würde. Ursprünglich wollte ich einfach schauen, ob der König aktuell im Schach steht und dann nur jene Züge erlauben, die aus dem Schach gehen, die Schachgebende Figur schlagen oder zwischen das Schach gehen. Diese Idee hat jedoch zwei Probleme. Einerseits muss ich immernoch überprüfen, ob der König nach einem Zug im Schach steht. Andererseits schaut es Fesselungen auch nicht an. Somit könnte man sich auch überlegen, Fesselungen separat anzuschauen. Dabei könnte mach für jede Figur eine Tabelle machen, die nach einer gegnerischen Figur weiter geht bis zur nächsten. Dies hat jedoch das Problem, dass es nicht alle Fesselungen findet, wie etwa in dieser Stellung:



Der letzte Zug von schwarz war Bauer von b7 nach b5. Dies würde normalerweise en-passent erlauben. Da danach jedoch die fünfte Reihe geräumt würde, geht dies nicht. Allerdings würde die oben genannte Funktion diesen Fall nicht finden. Zwar könnte man jene Fälle auch abdecken, jedoch könnte es noch weiter Ausnahmefälle geben, sodass ich einfach jeden Pseudo-legalen Zug durchführen muss und danach überprüfen muss, ob der König im Schach steht.

25.07.26

Samstag, 25. Juli 2026 20:37

Ich heute die Zuggeneration so weit gebracht, dass ich für eine gegebene Figur alle legalen Züge anzeigen kann. Mit Schachs. Somit kann ich schon bald den ersten PERFT machen. Bei einem solchen Test gibt man eine Stellung ins Programm ein. Dieses generiert dann alle legalen Stellungen bis zu einem bestimmten Punkt und gibt dann die Anzahl aus. Dabei kann man das eigene Resultat mit dem vorbestimmten vergleichen.
https://www.chessprogramming.org/Perft_Results
Ich werde vermutlich ein paar von diesen heraussuchen

26.07.26

Sonntag, 26. Juli 2026 20:29

Heute habe ich es geschafft, alle aus einer Stellung resultierenden legalen Stellungen zu generieren. Dazu habe ich auch versucht eine höhere Tiefe hinzukriegen, jedoch führt das zu einem error.

27.07.26

Montag, 27. Juli 2026 23:09

Ich habe heute den ersten erfolgreichen perft durchgeführt. Dafür musste ich eine Funktion bauen, die dies kann. Allerdings habe ich es nur bis zu perft(4) in der Startstellung geschafft. Perft 5 schaffte ich nicht, da das Programm extrem langsam war. Da ich den Grund dafür nicht verstanden habe, musste ich ChatGPT konsultieren. Ursprünglich dachte ich, dass es ein Problem mit VS code sein könnte. Somit habe ich das Programm in einem anderen Terminal laufen lassen, was jedoch zum selben Resultat führte. Nach weiter rüchsprache kam heraus, dass der Grund dafür darin liegt, dass ich jede Stellung nicht nur anschaue, sondern auch gleich abspeichere. Dies dauert extrem lange. Somit werde ich versuchen müssen, dies umzuprogrammieren.

https://chatgpt.com/share/6a67cb07-fd88-83eb-a01f-784cdb95a5f6

28.07.26

Dienstag, 28. Juli 2026 21:15

Heute habe ich das gesamte Programm praktisch neu machen müssen. Der Grund dafür ist, dass ich von ChatGPT herausgefunden habe, dass es Sinn ergibt, alle Vektoren von Figuren sowie dessen legalen Züge zusammen mit dem Brett zu speichern, sodass das Brett dann ein Objekt ist. Somit musste ich in jeder Funktion nochmals alles umprogrammieren, sodass es stimmt. Dazu bin ich aktuell daran, den verbesserten perft zu implementieren. Dabei geht es darum, dass ich die Stellungen nicht einzeln Speichere, sondern sie einzeln ausführe und die Züge dann wieder zurücknehme.

29.07.26

Mittwoch, 29. Juli 2026 19:30

Ich habe heute versucht den performance test nochmals neu zu machen nach dem Modell Von ChatGPT. Jedoch war ich nach ein paar Stunden sehr verzweifelt, da ich konstant bugs hatte. Als ich diese dann lösen konnte stellte sich heraus, dass das neue Modell langsamer ist als das vorherige. Das Problem dabei ist, dass es extrem langsam war. Sodass ich aus frustration Claude mein Programm gegeben habe und gesagt habe, es solle mir einen perft schreiben. Dieser ging jedoch auch nicht, sodass ich diesen debuggen musste. Als er dann ging war er deutlich schneller als seine Vorgänger, jedoch immer noch nicht so schnell, dass ich rasch meine Zuggeneration debuggen konnte. Ich habe noch weitere performance Verbesserungen von Claude übernommen, die das Programm schneller machen sollten. Jedoch ist es immernoch ziemlich langsam. Allerdings ist performance aktuell noch nicht mein Hauptanliegen, sondern die Zuggeneration. Somit werde ich mit dieser verminderter Performance dennoch perfts durchführen müssen.

https://claude.ai/share/414c105c-4e39-41c0-802a-771e7e860274

30.06.26

Donnerstag, 30. Juli 2026 23:18

Heute habe ich vorallem perfts gemacht und dann das Programm debugged. Es fing gut an und ich konnte schnell viele Bugs entferenen, jedoch ging dies nur bis zu einem gewissen Punkt gut, da auf einmal die Könige Schachs komplett ignoriert haben. Ich habe bis jetzt diesen Bug noch nicht patchen können, jedoch bin ich mir ziemlich sicher, dass dessen Ursache das fehlende Anpassen der pseudolegalen Züge ist.

31.07.26

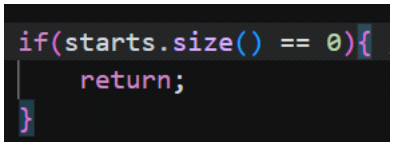
Freitag, 31. Juli 2026 22:55

Ich konnte den Bug immernoch nicht beheben. Dafür konnte ich einen Bug mit falschen en-passents beheben. Ich habe zudem herausgefunden, dass die Ursache des Bugs ein Doppelzug von Weiss ist. Was jedoch interessant ist ist, dass dieser nur auftaucht bei einer Tiefe von 5, wobei dieselbe Stellung jedoch auch bei Tiefe 4 existiert.

1.08.26

Samstag, 1. August 2026 23:18

Ich habe die Ursache für den Bug gefunden. Das Problem war, dass bei Stellungen ohne legale Züge (Schachmatt oder Patt) trotzdem manche Funktionen ausgeführt wurden. Somit konnte ich diese drei Linien einschieben



Welche das Problem komplett beheben konnten. Es ist irgendwie Ironisch, dass ein Bug, der mich für nun fast vier Tage geplagt hat, so eine leichte Lösung hat. Ich werde noch ein paar perfts durchführen. Falls diese dann fehlerfrei durchgehen kann ich dann mit meiner eigentlichen Arbeit starten. Für die perfts benutze ich folgende Website:
https://www.chessprogramming.org/Perft_Results

9.08.26

Sonntag, 9. August 2026 21:05

Heute habe ich weitere performance tests durchgeführt. Dabei konnte ich einen Bug erwischen. Dieser war, dass der König rochieren konnte selbst wenn er im Schach steht. Das darf er jedoch nicht, sodass es zu viele Stellungen gegeben hat.

`rnbq1k1r/pp1Pbppp/2p5/8/2B5/8/PPP1NnPP/RNBQK2R w KQ - 1 8 `

Depth	Nodes
1	44
2	1,486
3	62,379
4	2,103,487
5	89,941,194

Ich habe diese Stellung benutzt zum debuggen. Interessanterweise wurde in der Kopfzeile angemerkt, dass bei vielen Engines bei Tiefe 3 Bugs entdeckt wurden. So offenbar auch bei mir. Obwohl ich einen Bug beheben konnte, gibt es mir immer noch nicht das richtige Resultat zurück. Stattdessen gibt es mir frustrierenderweise folgendes Resultat zurück:

```
62378
```

Da ich keine Ahnung hatte wie ich diese einzelne Stellung isolieren konnte ging in zu einer anderen Stellung, um dort mein Glück zu versuchen.

Aus: https://chessprogramming.org/Perft_Results#fn11

`r2q1rk1/pP1p2pp/Q4n2/bbp1p3/Np6/1B3NBn/pPPP1PPP/R3K2R b KQ - 0 1 `

Depth	Nodes	Captures	E.p.	Castles	Promotions	Checks	Checkmates
1	6	0	0	0	0	0	0
2	264	87	0	6	48	10	0

Ich habe mit dieser Stellung dann weitergemacht. Es gab mir zwar das Richtige Resultat für Tiefe 1, jedoch gab es mir folgendes Resultat bei Tiefe 2:

Die Differenz zwischen der korrekten Lösung und meiner Lösung ist per Zufall die selbe Zahl wie die Anzahl Rochaden sowie die Anzahl vorangehenden Stellungen. Somit wird es etwas mit einem der beiden zu tun haben.

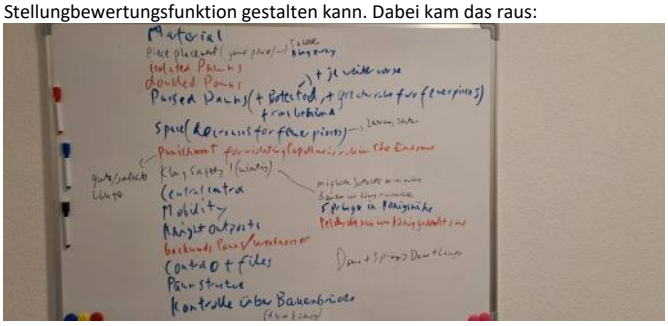
```
270
```

11.08.26

Dienstag, 11. August 2026 11:33

https://chessprogramming.org/Perft#divide

Ich bin nun soweit, dass mein Programm keine Fehler mehr macht mit der Stellungsgeneration. Soweit ich weiss. Ich konnte heute nämlich viele Bugs finden dank einer neuen Bugfinde Methode, die ich entdeckt habe. Die Idee dahinter ist, dass man die gefundenen Stellungen in der Startstellung nach der darauffolgenden Zügen unterteilt. Dies tut man zweimal. Einmal für die eigene Engine und einmal mit einer Engine, bei der man weiss, dass es klappen wird. Ursprünglicherweise wollte ich dafür Python verwenden. Python hat nämlich eine eingebaute Schach Bibliothek, womit dies möglich gewesen wäre. Jedoch konnte ich Python aus irgendeinem Grund Python nicht in VS Code implementieren. Allerdings habe ich herausgefunden, dass Claude diese Divides auch für einen machen kann. Somit kann ich nun eine Schachpartie spielen mit meinem Programm, wobei er zufällige Züge macht.
Während die perfts am laufen waren war ich am brainstormen, wie ich die Stellungbewertungsfunktion gestalten kann. Dabei kam das raus:



Blau sind Faktoren, die sich positiv auf die Stellungsbewertung auswirken, während rot jene sind, die sich negativ auswirken. Schwarz steht für Konzepte, die an sich nicht unbedingt positiv oder negativ sein müssen. Obwohl dies eine sehr grosse Liste ist, ist sie warscheinlich immer noch nicht komplett. Allerdings ist es auch eine ein wenig übertrieben grosse Liste. Für die erste Version der Stellungsbewertungsfunktion werde ich vermutlich nur das Material berücksichtigen.

https://claude.ai/share/538dbeda-f7b3-4487-835c-0d952cac7558
https://claude.ai/share/959c6c34-f362-4533-b3df-e5f0133a0ea4

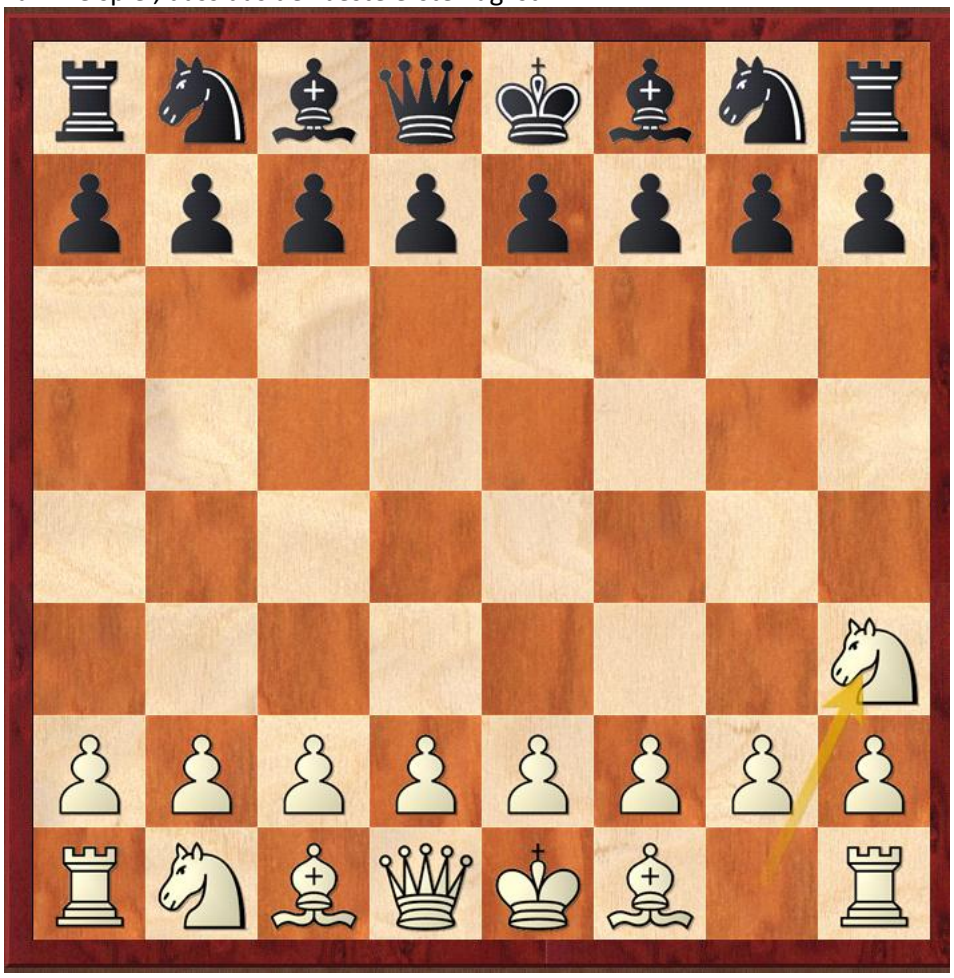
- Material
- Piece-Square Tables
- Pawn Structure
- Evaluation of Pieces
- Evaluation Patterns
- Mobility
- Center Control
- Connectivity
- Trapped Pieces
- King Safety
- Space
- Tempo

Aus:
https://chessprogramming.org/Evaluation

13.07.26

Donnerstag, 13. August 2026 23:57

Ich habe heute eine erste Version der Stellungsbewertung sowie eines Suchalgorthmuses gemacht. Die Stellungsbewertung kann bis jetzt das vorhandene Material zählen sowie erkennen, ob es Schachmatt ist. Dazu kann ich den besten Zug in kleinen Tiefen finden. Da es bislang allerdings nur das Material zählen kann, kann das Programm noch keine vernünftigen Züge machen. So denkt es zum Beispiel, dass das der beste erste Zug ist:



Vermutlich hat es einfach irgeneinen Zuf aus der Liste ausgewählt, da keiner etwas verliert- Allerdings hat das Programm schon enorme Mühe mit Tiefe 3. Dies liegt warscheinlich daran, dass ich in der Evaluationsfunktion die gesamte Stellung noch einmal neu auswerten lasse. Warscheinlcuh liegt es jedoch daran, dass ich momentan alle Züge durchgehe. Sobald ich dann das Pruning eingebaut habe, wird es hoffentlich schneller gehen.

14.08.26

Freitag, 14. August 2026 17:06

https://claude.ai/share/ae4a0eab-fafd-4a81-831b-82454c2c4616

https://chessprogramming.org/Alpha-Beta

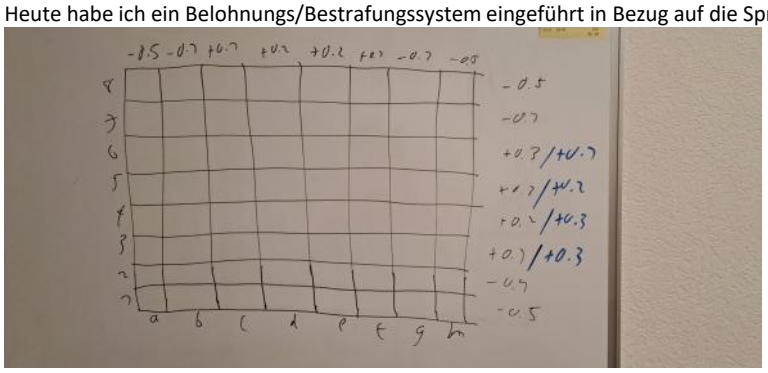
Heute habe ich eine erste Version des alpha-beat Prunings eingefügt. Es ist noch keine gute Version. Vor allem habe ich viel dafür leiden müssen. Selbst die Ratschläge von Claude verstand ich nur so halb. Das Ding beim alpha-beta Pruning ist vor allem, dass es darauf ankommt, wie schnell das Programm auf den besten Zug stösst. Somit ist eine gute Zugreihenfolge enorm wichtig. Momentan habe ich die Reihenfolge soweit, dass ich zuerst Damenzüge anschaue, dann Turmzüge, dann Läuferzüge usw. in der Reihenfolge der Figurenwerte. Zu den Figurenwerten: Ich habe den Läufer als 3.5 Punkte eingeschätzt. Normalerweise setzt man ihn auf 3, allerdings ist heutzutage das allgemeine Einverständnis, dass Läufer besser sind als Springer. Soweit ich weiss war es Fischer, der zuerst den Läufer auf den Wert von 3.5 gesetzt hat. Allerdings habe ich keine Quelle für das. Natürlich gibt es auch Stellungen, wo Springer besser sind als Läufer. Bei Schachcomputern arbeitet man jedoch meistens mit allgemeinen Konzepte, sodass ich denke, dass dieser Wert gerechtfertigt ist.

Ich habe auch die Stellungsbewertungsfunktion ein wenig erweitert. Es gibt nun eine kleine Belohnung, wenn ein Bauer im Zentrum steht. Um die Engine zu testen habe ich ein paar Partie gegen sie gespielt. In einer Dieser Partien brachte schwarz kurioserweise den König raus, nachdem es einen Bauern ins Zentrum gesetzt hat.

18.08.26

Dienstag, 18. August 2026 23:06

Heute habe ich ein Belohnungs/Bestrafungssystem eingeführt in Bezug auf die Springer.



Die Punkte sind dabei abhängig von der Reihe und der Linie. Dabei ist der Wert höher, je näher am Zentrum der Springer ist und je weiter er in der gegnerischen Stellung ist. Diese Werte sind für einen weissen Springer gedacht. Die schwarzen Springer haben leicht andere Werte, wie man rechts in blau sieht.

Dazu habe ich auch die Reihenfolge, mit der das Programm die Züge anschaut angepasst. Dabei gehe ich auf gewinnende Schlagzüge, gleiche Schlagzüge, verlierende Schlagzüge und Schachs in dieser Reihenfolge ein. Jedoch ist mir beim testen ein Bug aufgefallen, der vermutlich etwas mit dem Alpha-Beta Pruning zu tun hat. Somit werde ich dieses noch einmal überarbeiten müssen.

https://chessprogramming.org/Move_Ordering

19.08.26

Mittwoch, 19. August 2026 20:58

Ich konnte das Problem im Alpha Beta lösen soweit ich weiss. Dazu habe ich eine Belohnung eingeführt für Mobilität, also wie viele legal Züge eine Seite hat. Allerdings hat es dann einfach Damenzüge gemacht, da diese zu vielen neuen Zügen führen. Somit habe ich den Wert von Damenzüge halbiert. Gleichzeitig wollte ich die Entwicklung von Läufern fördern, indem ich jeden möglich Läuferzug um 0.2 Belohne. Allerdings muss ich warscheinlich dennoch für alle Figuren eine Feldspezifische Tabelle machen wie bei Springern. Jedoch müsste diese für manche Figuren je nach den Anzahl Figuren auf dem Brett angepasst werden. Der König steht im Mittelspiel nämlich am besten nahe an den Rändern. Im Endspiel hingegen steht er am besten aktiv.

20.08.26

Donnerstag, 20. August 2026 21:40

Heute ist mir ein Bug aufgefallen. Wenn die Engine nach Zügen sucht, geht sie zuerst in die Maximale Tiefe und geht dann langsam rückwärts. Mein Problem war, dass es Matts, die nicht in der maximalen Tiefe waren, nicht findet. Somit habe ich eine Ergänzung gemacht bei dem Teil, der Abbricht, wenn es keine legalen Züge gibt. Dazu habe ich noch Patts versucht einzuführen. Dies funktionierte in fast allen Fällen. Ausser bei einem. Bei dem sieht es zwar korrekterweise, dass es ein Patt gibt, führt jedoch den falschen Zug aus. Ich denke, dass das Problem irgendetwas mit den Positionen auf dem eval_tree zu tun hat.

21.08.26

Freitag, 21. August 2026 22:34

https://claude.ai/share/152b7594-a8cf-4ae2-bcd5-ee31cef38ff5

Heute habe ich die Hilfe von Claude beansprucht, um mein Programm zu verbessern. Das Problem war nämlich, dass meine Engine extrem langsam war. Es schaffte gerade einmal 60-70N/s. Nun schafft es ca. 1400N/s. Dies liegt daran, dass meine Funktion, um zu prüfen, ob nach einem Zug ein Schach ist, extrem ineffizient war. Ich kopierte nämlich das gesamte Brett mitsamt all seinen Vektoren und machte zudem alle Figurenlisten noch einmal von vorne. Dabei wäre dies komplett unnötig gewesen. In dieser neuen Funktion von Claude benötigt die Funktion nur den Vektor, also das eigentlich Brett, um zu schauen, ob es ein Schach gibt. Nachdem habe ich meinen Bot gegen Martin spielen lassen. Martin ist ein Bot auf Chess.com, der ziemlich bekannt dafür ist, schlecht zu sein. Mein Bot schaffte es zwar jedesmal eine gewonnene Stellung zu kriegen, jedoch schaffte er es nie, diese zu verwerten. Selbst wenn er 30 Materialpunkte mehr hatte. Dafür hatte ich zwei Lösungen:

1.

```
if(pieces <= 6){ //maximiert Königsmobilität im Endspiel
    eval += onecounter(board.whiteKing[0].legal_destinations);
    eval -= onecounter(board.blackKing[0].legal_destinations);
}
if(eval > 8){
    eval += 8-onecounter(board.blackKing[0].legal_destinations); //bei gewonnen Ste
    if(pieces <= 4){ //Bei gewonnenen Endspielen sollte der Abstand zum gegnerische
        eval -= sqrt(((board.whiteKing[0].square / 8) - (board.blackKing[0].square
        eval += abs(board.blackKing[0].square / 8 - 4) + abs(board.blackKing[0].squ
    }
}
if(eval < -8){
    eval -= 8-onecounter(board.whiteKing[0].legal_destinations);
    if(pieces <= 4){
        eval += sqrt(((board.whiteKing[0].square / 8) - (board.blackKing[0].square
        eval -= abs(board.blackKing[0].square / 8 - 4) + abs(board.blackKing[0].squ
    }
}
```

In diesem Codeausschnitt sollte im Endspiel die Mobilität des eigenen Königs maximiert werden während jene des Gegners minimiert werden sollte. Ist zudem die Stellung gewonnen, sollte der gegnerische König in die Ecke gedrängt werden während der eigene König näher kommen sollte. Mithilfe dieses Ausschnitts schafft es mein Programm, König gegen König + Turm zu gewinnen.

2.

```
void cycle(Board &board, int depth){
    int positions;
    double alpha = 10000;
    double beta = -10000;
    for(int i = 0; i < depth; i += 2){
        Tree(board, i + 2, positions, i + 2, alpha, beta);
        if(board.turn && board.eval == 1000){
            break;
        }
        if(!board.turn && board.eval == -1000){
            break;
        }

    }
}
```

Diese Funktion macht es, dass es alle Tiefen durchsucht bis zur gewünschten Tiefe. Dies ist einerseits nützlich, wenn ich die Zugreihenfolge verbessere. Andererseits ist es nützlich, um Matts in kleineren Tiefen als der maximalen zu finden. Ich habe ausserdem den loop so eingestellt, dass er nur in geraden Tiefen sucht. Dies ist wichtig, da dann die Suche auf einen gegnerischen Zug endet. Endete er auf einen eigenen, hätte das Programm das Gefühl, es könnte einfach irgendetwas machen, da es am Ende einfach etwas nehmen könnte. Dabei realisiert es nicht, dass der Gegner zurücknehmen kann, da es nicht so weit schaut.

Ich habe noch vesucht, ob die Engine mit Springer Läufer matt setzten kann. Dies konnte er nicht. Der Grund dafür ist, dass bei Springer läufer wichtig ist, in welcher Ecke der König gedrängt wird. Er sollte dabei in jene gedrängt werden, die der Läufer angreifen kann. Da dies die Engine jedoch nicht versteht, kann er die Stellung nicht gewinnen.

[Event "Play vs Bot"]
[Site "Chess.com"]
[Date "2026.08.21"]
[Round "-"]
[White "Martin-BOT"]
[Black "Profesore123"]
[Result "0-1"]
[CurrentPosition "r7/ppp2kpp/4p3/6P1/8/P7/2q5/K2r4 w - - 1 29"]
[Timezone "UTC"]
[ECO "C21"]
[ECOUrl "https://www.chess.com/openings/Center-Game"]
[UTCDate "2026.08.21"]
[UTCTime "22:02:06"]
[WhiteElo "250"]
[BlackElo "2314"]
[TimeControl "?"]
[Termination "Profesore123 won by checkmate"]
[StartTime "22:02:06"]
[EndDate "2026.08.21"]
[EndTime "22:22:28"]
[Link "https://www.chess.com/analysis/game/computer/1988869778/review?move=0"]
[WhiteUrl "https://images.chesscomfiles.com/uploads/v1/user/66746160.e1f80ccd.50x50o.8736a99052e0.png"]
[WhiteCountry "28"]
[WhiteTitle ""]
[BlackUrl "https://www.chess.com/bundles/web/images/noavatar_l.84a92436.gif"]
[BlackCountry "133"]
[BlackTitle "FM"]

1. e4 e5 2. d4 Nf6 3. dxe5 Nxe4 4. f3 Qh4+ 5. g3 Nxg3 6. hxg3 Qxh1 7. Qd5 Qxg1 8. Qxf7+ Kxf7 9. Ke2 Nc6 10. Be3 Nd4+ 11. Bxd4 Qxd4 12. g4 Bb4 13. e6+ dxe6 14. a3 Bc5 15. Ke1 Qf2+ 16. Kd1 Qxf1+ 17. Kd2 Rd8+ 18. Kc3 Bd7 19. f4 Qxf4 20. b4 Bd4+ 21. Kb3 Bxa1 22. Nc3 Qe3 23. Ka2 Qxc3 24. b5 Bxb5 25. g5 Bc4+ 26. Kb1 Ba2+ 27. Kxa2 Qxc2+ 28. Kxa1 Rd1# 0-1

22.08.26

Samstag, 22. August 2026 12:22

[Event "Play vs Bot"]
[Site "Chess.com"]
[Date "2026.08.22"]
[Round "-"]
[White "Profesore123"]
[Black "Santiago-BOT"]
[Result "1-0"]
[CurrentPosition "k7/2KS/RSP1/p7/p7/P7/8/8 b - - 4 72"]
[Timezone "UTC"]
[ECO "A00"]
[ECOUrl "https://www.chess.com/openings/Van-t-Kruijs-Opening"]
[UTCDate "2026.08.22"]
[UTCTime "09:43:01"]
[WhiteElo "2314"]
[BlackElo "850"]
[TimeControl "?"]
[Termination "Profesore123 won by checkmate"]
[StartTime "09:43:01"]
[EndDate "2026.08.22"]
[EndTime "10:21:04"]
[Link "https://www.chess.com/analysis/game/computer/1991368640/review?move=142"]
[WhiteUrl "https://www.chess.com/bundles/web/images/noavatar_l.84a92436.gif"]
[WhiteCountry "133"]
[WhiteTitle "FM"]
[BlackUrl
"https://images.chesscomfiles.com/uploads/v1/user/66746118.96f82f2a.50x50o.914a26e4bfb2.png"]
[BlackCountry "32"]
[BlackTitle ""]

1. e3 f5 $6 2. Nc3 e6 3. Bb5 $2 a5 4. Qh5+ $6 g6 $1 5. Qh3 Be7 $6 6. Kd1 $2 g5 7. Qh5+ Kf8 8. Nf3 $6 Ba3 $4 9. bxa3 d6 10. Qxg5 Qxg5 11. Nxg5 Kg7 12. Bb2 Bd7 $2 13. Nd5+ e5 14. Bxd7 Nxd7 15. Nxc7 Ne7 $2 16. Nxa8 Rxa8 17. Ne6+ Kf7 18. Ke2 Kxe6 19. f4 exf4 20. exf4 Re8 21. c4 d5 22. a4 $6 Nc6 $6 23. Be5 $6 Ndxe5 24. cxd5+ Kxd5 25. fxe5 $1 Nd4+ 26. Kf2 h6 27. Rae1 b5 28. Re3 Rxe5 29. Rxe5+ Kxe5 30. Re1+ Kd5 31. Re8 f4 32. g4 fxg3+ 33. hxg3 bxa4 34. g4 Ne6 35. d3 Kd6 36. Rh8 Ke5 37. Re8 Kd5 38. Rc8 Kd4 39. Rc6 Nc5 40. Kg2 Nb7 41. Rxh6 Kxd3 42. a3 Ke4 43. Rc6 Ke5 44. Rc2 Nd6 $2 45. Rc7 Nb5 $2 46. Rb7 $6 Kf6 $2 47. Rxb5 Kg6 $6 48. Rf5 Kg7 49. Rf4 Kg6 $6 50. Kg3 Kh6 51. Rf6+ Kg5 52. Rf7 Kg6 53. Rf8 Kh7 54. Rf6 Kg7 55. g5 Kh7 56. Rb6 Kg7 57. Rb7+ Kg8 58. Kf4 Kf8 59. Ke5 Ke8 60. Kf6 Kd8 61. Ke6 Kc8 62. Rd7 Kb8 63. Kd6 Ka8 64. Rc7 Kb8 65. Re7 Kc8 66. Re8+ Kb7 67. Rd8 Ka6 68. Rb8 Ka7 69. Kc7 Ka6 70. g6 Ka7 71. Rb6 Ka8 72. Ra6# 1-0

Heute habe ich meine Engine gegen diverse Bots auf Chess.com spielen lassen. Die Partien, die ich hier eingefügt habe, sind seine besten. Dabei habe ich immer korrekturen gemacht, wenn mir etwas nicht gefallen hat. Zu diesen korrekturen gehören: Formel für den Wert von Freibauern, Anpassung des Bonuses für Bauern im Zentrum, anpassung der Belohnung für Läufermobilität und viele weitere kleine Anpassungen. Der stärkste Bot, den meine Engine bis jetzt schlagen konnte hatte 1400 elo. Dabei machte er auch schon zwei Remis gegen jene mit 1500 Elo. Somit denke ich, dass man ihn auf ca. 1450 Elo einschätzen kann.

[Event "Play vs Bot"]
[Site "Chess.com"]
[Date "2026.08.22"]
[Round "?"]
[White "Hans"]
[Black "Profesore123"]
[Result "0-1"]
[BlackElo "2528"]
[WhiteElo "1000"]
[Termination "durch Schachmatt"]
[ECO "C45"]
[EndDate "2026.08.22"]
[Link "https://www.chess.com/game/computer/1992387512"]

1. e4 {Ah, die schönen Eröffnungsakkorde eines Liedes sind die gleichen wie die ersten Züge der Partie.} 1... e5 {Behalten wir den Spaß an der Sache $1} 2. Nf3 {Ich bin der Champion, mein Freund $1} 2... Nc6 3. d4 {Ich frage mich, ob Freddy Mercury Schach mochte $2} 3... d5 {Ich werde weiterkämpfen, bis zum Ende $1} 4. Bd3 dxe4 {Alle Songs müssen mit einer tollen Hook beginnen, so wie alle guten Spiele mit einem guten Schlag beginnen müssen $1} 5. O-O exd3 6. Qxd3 Be6 7. Nxe5 Nxe5 8. dxe5 Qxd3 {Du hast mir gerade mein Mädchen weggenommen $1} 9. cxd3 {Danke für deine Lady ;)} 9... Rd8 10. g4 Rxd3 {Warum müsst ihr mir in die Suppe spucken $2} 11. b3 Bxg4 12. Ba3 Bxa3 13. Nxa3 Nf6 14. exf6 Kd7 15. fxg7 Rg8 16. Nc2 Rxg7 {Nichts ist wirklich wichtig ... Jeder kann es sehen ... Nichts ist wirklich wichtig ... Nichts ist wirklich wichtig, für mich.} 17. f3 Bxf3+ {Donnerschläge und Blitze – sehr, sehr beängstigend $1} 18. Kf2 Rg2+ 19. Ke1 Re2# {Gutes Spiel $1 Da beißt wohl noch einer ins Gras.} 0-1

[Event "Play vs Bot"]
[Site "Chess.com"]
[Date "2026.08.22"]
[Round "-"]
[White "Mateo-BOT"]
[Black "Profesore123"]
[Result "0-1"]
[CurrentPosition "8/3b2k1/5p2/3P4/5B1P/7K/4r3/6q1 w - - 4 53"]
[Timezone "UTC"]
[ECO "A40"]
[ECOUrl "https://www.chess.com/openings/Queens-Pawn-Opening-Rapport-Random-Variation"]
[UTCDate "2026.08.22"]
[UTCTime "13:41:41"]
[WhiteElo "1400"]
[BlackElo "2314"]
[TimeControl "?"]
[Termination "Profesore123 won by checkmate"]
[StartTime "13:41:41"]
[EndDate "2026.08.22"]
[EndTime "15:04:16"]
[Link
"https://www.chess.com/analysis/game/computer/1992751440/review?move=103"]
[WhiteUrl
"https://images.chesscomfiles.com/uploads/v1/user/66746062.968578ec.50x50o.b410e5352484.png"]
[WhiteCountry "4"]
[WhiteTitle ""]
[BlackUrl
"https://www.chess.com/bundles/web/images/noavatar_l.84a92436.gif"]
[BlackCountry "133"]
[BlackTitle "FM"]

1. d4 e6 2. c4 Nc6 3. d5 exd5 4. cxd5 Bb4+ 5. Nc3 Ne5 6. a3 $6 Bxc3+ $1 7. bxc3 d6
8. f4 $2 Nc4 $4 9. Qa4+ $1 b5 10. Qxb5+ Bd7 11. Qxc4 Ne7 12. e4 O-O 13. Rb1 Rb8 14.
Rb3 Rxb3 15. Qxb3 Qb8 16. Qxb8 Rxb8 17. Be3 $6 Rb1+ 18. Kf2 $1 Rb2+ 19. Be2 Rb1 20.
h3 a5 21. Kg3 f5 22. Bd3 Rd1 23. Bc2 Ra1 24. Bd3 Rd1 25. Bc2 Ra1 26. Kf3 $6 fxe4+
27. Bxe4 Kh8 $6 28. h4 h6 29. f5 h5 30. Bf4 $6 Re1 31. Bd2 Rf1+ 32. Ke2 $6 Ra1 $6
33. f6 gxf6 34. g4 $4 hxg4 35. c4 $6 Rxa3 36. Bc1 Ra2+ 37. Ke3 $6 g3 $2 38. Nf3 $6 c6
39. Nd4 $6 cxd5 40. Bxd5 Nxd5+ 41. Kf3 $2 Kg7 42. cxd5 a4 43. Ne2 $6 g2 44. Rh2 $4
g1=Q $1 45. Rg2+ Qxg2+ 46. Kxg2 Rxe2+ 47. Kf3 Bb5 48. Bf4 a3 49. Kg3 $6 a2 50. Bxd6
a1=Q 51. Bf4 Qg1+ 52. Kh3 Bd7# 0-1

25.08.26

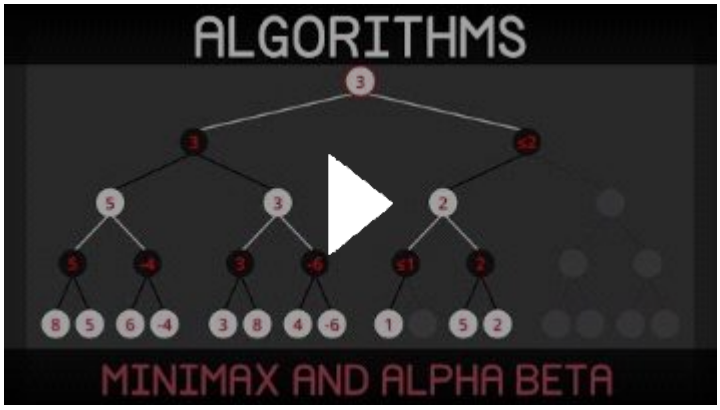
Dienstag, 25. August 2026 14:33

Heute habe ich eine Bedingung für Springervorpfosten gemacht. Ich muss sie vermutlich noch anpassen, da sie jene Felder als Vorpfosten sieht, bei denen der Gegner keine Möglichkeiten hat, um den Springer mit einem Bauer anzugreifen. Dies schliesst jedoch nicht aus, dass es dies im nächsten Zug kann.

27.08.26

Donnerstag, 27. August 2026 21:46

Heute habe ich das Alpha-Beta pruning nochmals komplett neu gemacht. Dafür habe ich folgendes Video als Vorlage benutzt: [Algorithms Explained – minimax and alpha-beta pruning]

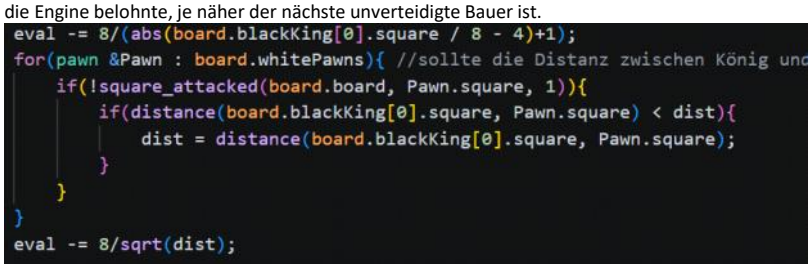


Nun ist das Programm schon deutlich schneller. Ich habe gemerkt, dass etws nicht stimmte, da die Enginge viele Züge nicht machen wollte, die es eigentlich sehen sollte. Dies lag daran, dass mein ursprüngliches alpha-beta pruning falsch gepruned hat und somit legitime Optionen gelöscht hat während es überflüssige Optionen behalten hat.

28.08.26

Freitag, 28. August 2026 18:20

Heute habe ich die Endspielfähigkeiten der Engine erweitert. Dies konnte ich machen, indem ich die Engine belohnte, je näher der nächste unverteidigte Bauer ist.



```
eval -= 8/(abs(board.blackKing[0].square / 8 - 4)+1);
for(pawn &Pawn : board.whitePawns){ //sollte die Distanz zwischen König und
    if(!square_attacked(board.board, Pawn.square, 1)){
        if(distance(board.blackKing[0].square, Pawn.square) < dist){
            dist = distance(board.blackKing[0].square, Pawn.square);
        }
    }
}
eval -= 8/sqrt(dist);
```

Dazu gebe ich ihm auch einen Bonus, je weiter oben der König ist. Den Wert habe ich unter die Wurzel genommen, damit es für grosse Distanzen nicht übertrieben gross werden würde und somit dennoch einen Einfluss auf die Evaluation haben kann. Allerdings macht die Engine immer noch blöde Fehler. Dabei konnte ich sie meistens auf fehlende Tiefe zurückführen. Aktuell kann ich nur mit Tiefe 4 in den meisten Stellungen rechnen. Das heisst zwei Züge pro Person. Ich habe dafür das Programm Claude gegeben, wobei rauskam, dass das Rückstellen von Stellungen durch das Speichern von FEN-codes extrem teuer ist. Ich werde somit als nächstes versuche, das zu verbessern.

https://claude.ai/share/b3ed8cc8-b9d1-4b43-a486-e91c734db0e0

29.08.26

Samstag, 29. August 2026 21:29

Heute habe ich die Undomove Funktion überarbeitet. Ich kann nun Tiefe 6 durchführen, wobei dies jedoch immernoch etwa eine Minute geht. Das Problem war, dass ich zuvor die Stellungen als FEN-codes gespeichert habe. Dies war zwar praktisch, jedoch war es extrem ineffizient. Nun habe ich alle notwendigen Informationen in ein Objekt gepackt.

30.08.26

Sonntag, 30. August 2026 21:45

Heute habe ich versucht, piecelist zu eliminieren indem ich die Listen in undomove und domove selber aktualisiere. Dies schien zuerst gut zu klappen und ich bin soweit, dass das Programm nicht mehr streikt. Jedoch gibt es mir nun unmöglich hohe Stellungsbewertungen wie etwa diese:
`-55.55`
Dies deutet darauf hin, dass das Programm voller Bugs ist, die ich beheben muss.

31.08.26

Montag, 31. August 2026 22:53

Heute konnte ich einige Bugs eliminieren. Darunter der Stellungsbewertungsbug von gestern. Aus irgendeinem Grund hat das Programm die Figurenwerte total inflatiert bei höheren Tiefen, was zu sehr hohen Stellungsbewertungen führte. Dazu habe ich heute git installiert, sodass ich einfacher Versionen abspeichern kann.

02.09.26

Mittwoch, 2. September 2026 22:46

Heute habe ich einige Optimisationen gemacht. Zuerst habe ich die Piecefinder Funktionen aufgeräumt, indem ich unnötige Parameter gestrichen habe und ein pass by reference eingefügt habe. Dazu habe ich versucht ein System einzufügen, wobei das Programm über die maximale Tiefe weiter rechnet, wenn es forciert ist. Dies schien gut zu funktionieren, jedoch geht es immer noch nicht wie ich es möchte.

Optimisationsideen:

https://chatgpt.com/share/6a988c04-e89c-83ed-9e45-d7c8ca83d2b1

03.09.26

Donnerstag, 3. September 2026 20:40

https://claude.ai/share/8a653e3d-2b04-4782-ac38-946ac781e8af

Problematische Stellung: r1bk2nr/p1p1ppbp/1pP5/1B4B1/8/5p2/PPP2PPP/RN2K2R w KQ - 0 9

Heute habe ich an der forcierten sucherweiterung weitergemacht. Ich habe hinzugefügt, dass das Programm weitersuchen soll, wenn der vorletzte Zug ein Schach war. Allerdings ging die Suche immernoch ziemlich langsam, sodass ich das Programm Claude gegeben habe, um es zu optimieren.

04.09.26

Samstag, 5. September 2026 00:41

Heute habe ich wieder die Engine an Bots ausprobiert. Dabei hat er allerdings einen ziemlich offensichtlichen Zug nicht gemacht. Dies habe ich dann untersucht und habe dann herausgefunden, dass er aufhört rechnen, wenn man kurz vor dem Matt ist, da es keine Anweisungen hat zum dann weitere rechnen. Somit bin ich nun an einem Matt detektor dran.

05.09.26

Sonntag, 6. September 2026 00:12

Heute habe ich den Matt-detektor fertig gemacht leider kann ich ihn jedoch nicht auf die Art benutzen die ich wollte, da es dann einen infinite loop geben würde. Dieser wäre: Mate_detection--> domove --> remember_board --> Mate_detection. Mir ist bis jetzt noch kein schlauer Weg eingefallen, wie ich es auf die gewünschte Art implementieren kann. Ich denke, dass ich dies für jetzt einmal stehen lassen werde und stattdessen meine Augen auf ein anderes Ziel setzen werde. Nämlich transposition tables. Ich denke, dass das das letzte grosse Ding ist, das ich noch am Bot ändern werde. Die Idee hinter dem transposition table ist es, schon gesuchte Stellungen mit ihrer Tiefe und Stellungsbewertung zu hashen und zu speichern. Diese kann man dann abrufen wenn eine identische Stellung erreicht wird. Bis lang habe ich folgende Quelle zu dem verfahren:

06.09.26

Sonntag, 6. September 2026 15:11

https://chatgpt.com/share/6a9d665b-79f8-83eb-bcf0-d3a5ba931e96

Heute habe ich verucht, einen transposition Table einzuführen. Dafür musste ich zobrist Hashing implementieren. Da dieses Konzept eher kompliziert ist, habe ich ChatGPT es mir erklären lassen sowie mich leiten lassen. Das Hashing scheint zu gehen, jedoch muss ich noch einen effizienteren Weg implementieren, um die gehashten Stellungen zu durchgehen.

12.09.26

Samstag, 12. September 2026 12:33

https://claude.ai/share/a34af638-9aa8-477a-a129-1d92ad327b72

Ich habe heute den Transposition_table mithilfe von Claude beenden lassen. Ob die Engine jedoch schneller ist als ohne, ist mir nicht ganz klar. IN manchen Situationen ist sie durchaus schneller, jedoch ist sie in manchen Situationen auch langsamer.

24.09.26

Mittwoch, 23. September 2026 22:13

Heute habe ich die dreifache Stellungswiederholungs Regel implementieren können. Dazu habe ich den Bot gegen einige Bots auf Chess.com getestet. Dabei gewann er gegen einen Bot mit 1800 Elo. Ich habe mir auch noch eine To Do Liste gemacht. Diese sieht so aus:

- 3-fache Stellungs Wiederholung
- Prorität zu schnelleren Siegen
- Bessere Königssicherheit
- 50 Züge Regel
- Zeit-Benchmark
- Versionen Benchmark
- Time-based cycling
- AI
- Viele Partien spielen
- Endspiele

Davon konnte ich schon das erste streichen.

25.09.26

Freitag, 25. September 2026 19:37

Heute habe ich die 50 Züge Regel eingeführt. Sie ist eigentlcih ziemlich unnötig, da es warscheinlich eh nie so weit kommen wird. Tatsächlich habe ich sie noch nie in einer meiner eigenen Partien erlebt. Dennoch habe ich sie eingeführt damit das Programm komplett ist. Dazu habe ich begonnnen mit einer besseren Königssicherheits Beschreibung. Diese sieht bislang so aus:

```
for(int i = 0; i < 3; i++){
    if(!(board.whiteKing[0].square % 8 == 0 && i == 0) && board.board[board.whiteKing[0].square - 9 + i] == 'P')
        eval += 0.3;
    }
}
```

Die Idee dahinter ist, dass ich die Felder vor dem König durchgehe und für jeden Bauer, der dort vorhanden ist, gibt man einen kleinen Bonus. Allerdings funktioniert dies nur, wenn der König rochiert ist.

5.10.26

Montag, 5. Oktober 2026 17:06

https://claude.ai/share/29bcead3-b959-4201-bdb4-dbd9bb1b92bf

Heute habe ich mit dem Schriftlichen Teil meiner Maturaarbeit weitergemacht. Ich hatte zuvor die Einführung geschrieben sowie den Abschnitt zur Motivation, habe aber vergessen dies zu dokumentieren. Es wäre am Donnerstag gewesen, also 1.10.26. Heute habe ich die Regeln zusammengefasst und auch eine kurze Geschichte zu Schachcomputern geschrieben. Die Einflüsse auf das menschliche Spiel habe ich begonnen nach meinem Wissen zu notieren, allerdings muss ich mein Wissen noch überprüfen. Ich habe auch meine Erklärungen zu den Schachregeln von Claude überprüfen lassen und musste sie schon ziemlich korrigieren.

6.10.26

Mittwoch, 7. Oktober 2026 00:40

Heute habe ich die Abschnitte zum minimax und zum alpha beta pruning geschrieben. Ich werde sie vermutlich nochmals überarbeiten morgen, da ich noch nicht ganz zufrieden bin mit ihnen

7.10.2026

Donnerstag, 8. Oktober 2026 00:55

Heute habe ich den theoretischen Teil fertig geschrieben und habe mit dem praktischen Teil begonnen. Dabei ist mir beim Teil mit der Stellungsbewertung eingefallen, dass ich gerne die einzelnen Bewerungskriterien nach ihrer Wichtigkeit anordnen würde. Somit musste ich einen Weg finden, um den Bot automatisch viele Partien spielen zu lassen. Zum Glück gibt es Programm mit denen man das machen kann. Ich benutze Cutechess. Dafür musste ich jedoch UCI einprogrammieren. Dieses habe ich nicht ganz verstanden, sodass ich es an Claude übergeben habe. Danach ist mir noch aufgefallen, dass ich ausversehen verschiedene Updates in zwei verschiedenen Dateien gemacht habe. Um dies zu lösen habe ich Claude benutzt, um die beiden Programme zu 'fusionieren'.

```
$cli = Get-ChildItem "C:\Program Files","C:\Program Files (x86)",$env:USERPROFILE -Recurse -Filter cutechess-cli.exe -ErrorAction SilentlyContinue | Select-Object -First 1 -ExpandProperty FullName
if (-not $cli) { Write-Host "cutechess-cli.exe not found" -ForegroundColor Red; return }
if (-not (Test-Path 8moves_v3.pgn)) {
  Invoke-WebRequest https://github.com/official-stockfish/books/raw/master/8moves_v3.pgn.zip -OutFile 8moves_v3.pgn.zip
  Expand-Archive 8moves_v3.pgn.zip -DestinationPath . -Force
}
& $cli -engine conf=Chess_new_1_w_piecevalue -engine conf=Jobot1 -each tc=10+0.1 -openings file=8moves_v3.pgn format=pgn order=random plies=16 -rounds 5 -games 2 -repeat -concurrency 4 -ratinginterval 20 -recover -pgnout results.pgn
```

8.10.26

Donnerstag, 8. Oktober 2026 11:11

https://claude.ai/share/4f3b2325-e80c-4dd5-87aa-211236ba6721

https://claude.ai/share/4f3b2325-e80c-4dd5-87aa-211236ba6721

https://claude.ai/share/a153cc18-cf83-4aa3-a4aa-e2be720e79b8

Optimierte Version: https://claude.ai/share/45bd4479-da3d-4d48-aed0-b1659ea45f19

```
Wait-Process -Name cutechess-cli; & ".\cutechess-1.5.1-win64\cutechess-1.5.1-win64
\cutechess-cli.exe" -engine name=BotA cmd=chess_new_1_opt.exe -engine name=BotB
cmd=chess_bunny_unde.exe -each proto=uci tc=inf depth=4 -openings file=book
\8moves_v3.pgn format=pgn order=random -rounds 500 -games 2 -repeat -concurrency
$env:NUMBER_OF_PROCESSORS -maxmoves 200 -recover -ratinginterval 100 -pgnout
selfplay_d4.12.pgn
```

Durchzuführende Tests:
Ohne jedes Bewertungskriterium,
Ruhesuche Tiefe 4 vs. Feste Suche Tiefe 4/6

4 = Vorpfosten, 1000 Partien, Tiefe 4
3 = Abstand gegnerischer König, 10000 Partien, Tiefe 2
2 = Königsmobilität des Gegners einschränken in gewonnenen Stellungen, 10000 Partien, Tiefe 2
1 = Springer Felder, 1000 Partien, Tiefe 4
0 = schwache Bauern. 10000 Partie, Tiefe 2

Heute war ein produktiver Tag. Ich habe so viel gescaafft, wich weiss gar nicht mehr richtig, was ich alels gemacht habe. Ein wichtiger Teil davon war, dass ich mir Claude premium geholt habe. Ich habe zuerst Claude benutzt, um mein Programm deutlich effizienter zu machen. Dabei hat es jeden einzelnen Fehler markiert. Dazu habe ich auch noch am schriftlichen Teil weitergemacht. Dabei ist mir eingefallen, dass es eine coole Idee wäre, die Wichtigkeit der einzelnen Bewertungsparameter zu untersuchen mit dem neu erworbenen Cutechess. Das habe ich auch gemacht. Dabei braucht mein laptop etwa eine halbe Stunde, um 1000 Partien zu spielen. Allerdings gibt es sogar einen noch besseren Weg, mit welchem man in derselben Zeit etwa 7000 Partien spielen kann(das ist bloss eine grobe abschätzung. Tatsächlich weiss ich es nicht). Bis lang hat jedoch nur das Material eine deutliche Auslenkung angezeigt. Die anderen Parameter scheinen keinen Unterschied zu machen. Allerdings liegt dies vermutlich nur daran, dass 1000 Partien zu wenig ist. Um nun allerdings die Relevanz der einzelnen Veränderungen zu berechenen musste ich eine kurze Sidequest zur normalen Verteilung machen. Zum Glück ist diese Jedoch im meinem Schulmathebuch erklärt drin, sodass ich sogar dieses als Quelle angeben konnte:).

Der Terminalcommand:

```
& ".\cutechess-1.5.1-win64\cutechess-1.5.1-win64\cutechess-cli.exe" -engine name=BotA
cmd=chess_new_1_opt.exe -engine name=BotB cmd=chess_new_1_opt -each proto=uci
tc=inf depth=2 -openings file=book\8moves_v3.pgn format=pgn order=random -rounds
5000 -games 2 -repeat -concurrency $env:NUMBER_OF_PROCESSORS -maxmoves 200 -
recover -ratinginterval 100 -pgnout selfplay_10k_.04.pgn
```

Stockfish:

```
& ".\cutechess-1.5.1-win64\cutechess-1.5.1-win64\cutechess-cli.exe" `
  -engine name=BotA cmd=".\chess_new_1_opt.exe" proto=uci tc=inf depth=4 `
  -engine name=stockfish-windows-x86-64-universal cmd="C:\Users\johan\OneDrive - kszi.ch\Schule
\Informatik\.vscode\chess\stockfish-windows-x86-64-universal.exe" proto=uci tc=120+1
option.UCI_LimitStrength=true option.UCI_Elo=1700 option.Threads=1 option.Hash=16 `
  -openings file=book\8moves_v3.pgn format=pgn order=random `
  -rounds 250 -games 2 -repeat `
  -concurrency 6 `
  -draw movenumber=40 movecount=8 score=10 `
  -resign movecount=4 score=1000 twosided=true `
  -maxmoves 200 -recover -ratinginterval 50 `
  -pgnout BotA_d2_vs_SF1500.pgn
```

12.10.26

Dienstag, 13. Oktober 2026 11:53

Heute habe ich weiter am schriftlichen Teil gemacht.

13.10.26

Mittwoch, 14. Oktober 2026 01:05

Heute habe ich weiter gemacht am schriftlichen Teil. Zudem habe ich eine rieseige Verbesserung im Endspiel vornehmen können. Diese ist mir aufgefallen, als ich den Bot an meinem Vater testete. Allerdings verlangsamt es das Programm etwas. Ich bin momentan noch dran, die letzten Stellungsbewertungskriterien zu testen. Gleichzeitig versuche ich einerseits das Elo des Bots herauszufinden, indem ich ihn gegen Stockfish auf 1500 Elo spielen lasse und dann die Elo differenz aufgrund des scores ausrechne. Andererseits versuche ich auch den Suchalgorithmus zu benchmarken um herauszufinden, welche Teile des Suchalgorithmus wie wichtig sind in der Geschwindigkeit.

14.10.26

Donnerstag, 15. Oktober 2026 02:29

Heute habe ich das benchmark Kapitel geschrieben. Bei den benschmarks habe ich dasselbe Verfahren verwendet wie bei der Stellungsbewertungsfunktion. Allerdings musste ich es einmal komplett von neu machen. Das war, da ich meinen Test selber sabotiert habe. Der Test misst die Geschwindigkeit des Programms. Dabei hatte ich noch viele andere tasks offen, was den Test verfälschte. Allerdings konnte ich ihn erneut machen und habe nun ein, meiner Meinung nach, cooles Kapitel geschrieben. Allerdings habe ich ein Problem. Währen dem programmieren habe ich einiges mit Claude machen lassen. Dabei habe ich die jeweiligen Chats als Link kopiert. Jedoch habe ich vergessen, beim Link die Erlaubnis zu erweitern. Normalerweise wäre dies kein Problem. Ich kann den Link einfach nochmals kopieren. Jedoch habe ich diesen Account vor einiger Zeit verloren, da ich ihn gemacht habe als ich noch unter 18 war. Er wurde deshalb von Anthropic gebannt.

15.10.26

Donnerstag, 15. Oktober 2026 20:55

Heute habe ich einerseits am schriftlichen Teil weitergemacht. Andererseits habe ich auch die Engine spielbar gemacht auf Lichess. Er heisst Jobot123.

29.10.26

Freitag, 30. Oktober 2026 07:38

Habe am schriftlihen Teil weiter gemacht, ein Github eingerichtet und testts and der Engine durchgeführt

30.10.26

Samstag, 31. Oktober 2026 11:39

Heute habe ich am schriftlichen Teil weitergemacht. Mir fehlt nur noch das Fazit und ein paar Tests für die erste Rohfassung.

31.10.26

Sonntag, 1. November 2026 03:03

Heute habe ich am schriftlichen Teil weitergemacht.

06.11.26

Samstag, 7. November 2026 02:50

Heute habe ich am schriftlichen Teil weiter gemacht. Dazu habe ich noch einen Bug gefunden in meinem Code bei der Freibauerevaluation.